RapidBoxForest: Link-Envelope-Guided Box Forests for Fast Multi-Query Manipulation

TIAN, Yu and Ren, Hongliang

***Abstract*—Repeated-query manipulation needs reusable free-space structure, yet constructing high-quality convex regions can be expensive in high-dimensional scenes and fragile under local edits. This paper presents RapidBoxForest (RBF), a link-envelope-guided box-forest planner for this low-build-cost regime. First, RBF repurposes endpoint-to-link swept-capsule bounds as a configuration-space box-validation oracle: endpoint interval envelopes over a joint box are radius-lifted into link envelopes, which validate volumetric regions rather than individual path segments. Second, accepted boxes are grown into a scene-specific forest, consolidated into a typed corridor graph, and queried by membership lookup plus graph search for repeated start-goal pairs. Third, a Lifelong Envelope Cache Tree stores kinematics-keyed envelope evidence separately from the scene-level cache of box labels and connectivity. The formal claim is soundness for paths contained in conservatively validated box unions; tighter filters, repair, and post-processing are counted only after the configured query-validation path succeeds. Under this validation policy, the selected Shelf+IIWA profile builds in 0.070 [0.070,0.071] s with 0.032 [0.031,0.032] s five-query amortized time after replaying a warm d23 evidence snapshot, and isolated local maintenance is 24.5–55.9× faster than fresh warm leaf-sweep rebuilds.**

***Index Terms*—Motion planning, multi-query planning, link interval envelopes, box-region planning, configuration-space planning.**

I. Introduction

High-DOF manipulators often face many related queries in nearly fixed workcells: the robot kinematics persist, while start-goal pairs, local obstacles, or small scene edits change. Reuse is valuable only if the reusable object is cheap enough to build, update, and query.

Convex-region methods such as IRIS and Graphs of Convex Sets provide powerful region-level planning abstractions once certified regions are available [1], [2], [3], but their construction cost can dominate in high-dimensional or frequently edited scenes. Sampling-based methods such as PRM and RRT variants avoid heavy preprocessing and solve single queries effectively [4], [5], [6], [7], yet their reusable structures are usually zero-volume roadmaps or trees.

RBF targets the middle ground: it grows axis-aligned C-space boxes, validates them with a link-envelope scene oracle, connects accepted boxes into a graph, and answers later queries by searching the resulting corridor. The design trades region expressivity and path optimality for low build cost, simple adjacency, and explicit multi-query reuse. The resulting planning object is intended for regimes in which repeated-query latency and rebuild cost matter more than obtaining a tighter regional decomposition.

a) Problem statement.: Let $\mathcal{Q} \subset \mathbb{R}^n$ be the joint-limit box of a fixed manipulator and let $\mathcal{O} \subset \mathbb{R}^3$ be the current workspace obstacle set. Given a batch or stream of start-goal queries $(q_s, q_g) \in \mathcal{Q}^2$, the objective is to build a scene-specific planning object that supports repeated query retrieval while amortizing geometric work that depends only on robot kinematics and joint intervals. The object sought by RBF is a finite family of validated C-space boxes and an adjacency graph over those boxes. Its construction should have lower rebuild and local-update cost than a high-quality convex-region decomposition, while still exposing volumetric free-space structure rather than only path-local collision checks.

The key geometric ingredient is not a new capsule swept-volume identity, but a new use of the classical endpoint-bounding pipeline. For rounded link models, RBF separates the swept capsule into the sweep of a zero-radius center segment and a final radius lift. Endpoint interval envelopes bound the two endpoint trajectories over a joint box; the convex hull of these endpoint envelopes, enlarged by the link radius, encloses the swept rounded link. RBF uses this bound as a C-space box free-space validation oracle. The same endpoint evidence can be exported as axis-aligned bounding boxes (AABBs), SupportHull records, or staged AABB–SupportHull filters. Conservative endpoint sources yield a conditional conservative box certificate, whereas tighter practical sources act as planning filters under the common query-validation policy.

The planning layer builds directly on this primitive. A sample-guided grower selects query-relevant frontiers, materializes local candidate boxes around projected seeds, and extends a connected forest of validated regions. This is a workload-biased adaptive cell decomposition in manipulator C-space. RBF has two reuse levels. The scene-level RBF cache is the constructed box forest and corridor graph for a fixed obstacle set; it directly answers many start-goal queries in that environment. The lifelong envelope cache, LECT, memoizes scene-independent endpoint

The authors are with the Department of Electronic Engineering, The Chinese University of Hong Kong (CUHK), Hong Kong SAR, China. E-mail: ty1997@link.cuhk.edu.hk.

TABLE I
Qualitative comparison of reusable manipulation-planning paradigms.

Family	Reuse object	Strength	Limitation	Relation to RBF
PRM / LazyPRM	Roadmap	Multi-query graph	Edge checks; zero volume	RBF reuses validated boxes rather than only edges.
RRTConnect / RRT	Query tree	Fast discovery	Little persistent reuse	RBF turns frontier growth into reusable box growth.
BIT* / FMT*	Sample graph	Anytime improvement	Query-centric checking	RBF prioritizes reusable low-cost regions.
IRIS / GCS	Convex regions	Optimization-ready graph	Costly inflation	RBF uses cheaper boxes instead of rich convex sets.
RBF	Box forest + LECT	Volume reuse	Conservative boxes	CCD-style link envelopes validate C-space boxes; scene and kinematic caches remain separate.

and link-envelope evidence keyed by robot kinematics and joint intervals; it accelerates future builds or local updates, while free-space labels remain scene-dependent.

The evaluation is organized around this design point. It examines repeated-query Shelf+IIWA planning, evidence replay, AABB-versus-SupportHull link-envelope representation trade-offs, and local maintenance under obstacle edits. The resulting claims are framed around build cost, multi-query reuse, envelope-representation trade-offs, and update behavior rather than around a universal ranking against all single-query planners. This scope is deliberate: one-shot planners can still be preferable for isolated queries or when path optimality dominates preprocessing cost. RBF is intended for repeated-query workcells where a lightweight reusable region graph is useful before heavier convex-region construction is justified.

RBF is therefore a planning-systems contribution enabled by a geometric primitive. The endpoint-to-link envelope supplies the C-space box-validation oracle; the scene-level box-forest cache, lifelong LECT evidence cache, and local update rules show how that oracle can be assembled into a reusable multi-query planning pipeline.

The paper contributes RBF**, a link-envelope-guided box-region planner that lifts classical swept-capsule endpoint bounds from collision detection to reusable C-space free-space construction.** The first item is the geometric/formal contribution, the next three are system contributions, and the last item is the evaluation contribution:

1) *Geometric primitive reuse: joint-box link-envelope validation.* RBF repurposes CCD-style endpoint bounds as a joint-box free-space oracle, with conservative sources for certificates and tighter staged sources for query-validated filtering (section III).
2) *Planning system: box forest and typed corridor graph.* Seed descent, leaf-sweep coverage, restricted refinement, frontier expansion, and consolidation produce a scene-level typed corridor graph of validated boxes (section V).
3) *Reuse architecture: scene cache versus LECT evidence cache.* Scene RBF caches store the current environment's box forest, while LECT stores lifelong kinematics-keyed endpoint and link-envelope evidence reusable across builds (sections IV and V).
4) *Local maintenance: deletion promotion and insertion refill.* Deletions revalidate formerly blocked domains for promotion, and insertions invalidate conflicting boxes before bounded local refill (section V-G).
5) *Evaluation protocol: saved catalogs and audit-separated timing.* The experiments separate build, online solve, final simplification, and strict audit time, keep saved scene/query catalogs fixed across planners, and report cache reuse only when the corresponding LECT evidence is explicitly enabled (section VI).

The rest of the paper develops the envelope oracle (section III), LECT reuse layer (section IV), box forest and update rules (section V), and evaluation (section VI).

II. Related Work

A. Convex Region Planning and Graphs of Convex Sets

Convex region planners approach motion planning by constructing a certified cover of free space and then solving graph search or convex optimization on top of that cover. IRIS introduced the now-standard idea of computing large collision-free convex regions by alternating separating hyperplanes and region inflation [1]. Later work extended the same line into configuration space, improved region quality, or improved graph compactness through visibility-graph and clique-cover constructions [8], [9], [10]. Marcucci *et al.* made the Graph-of-Convex-Sets formulation particularly influential by showing that motion planning can be cast as optimization over a graph of certified convex regions [2], [3].

RBF shares the region-level objective of this literature but uses a different primitive. Its regions are axis-aligned boxes in C-space rather than general polytopes, so the cover is less expressive than a high-quality convex decomposition. The payoff is a lower-cost validation loop, simple adjacency bookkeeping, and direct local maintenance. The relevant comparison is therefore not whether boxes are intrinsically better than polytopes; it is whether a lower-cost region primitive can support repeated-query planning before the cost of tighter convexification is justified.

B. Swept Volume Approximation and Capsule Geometry

Computing the swept volume of a moving rigid body has been studied extensively in computational geometry and

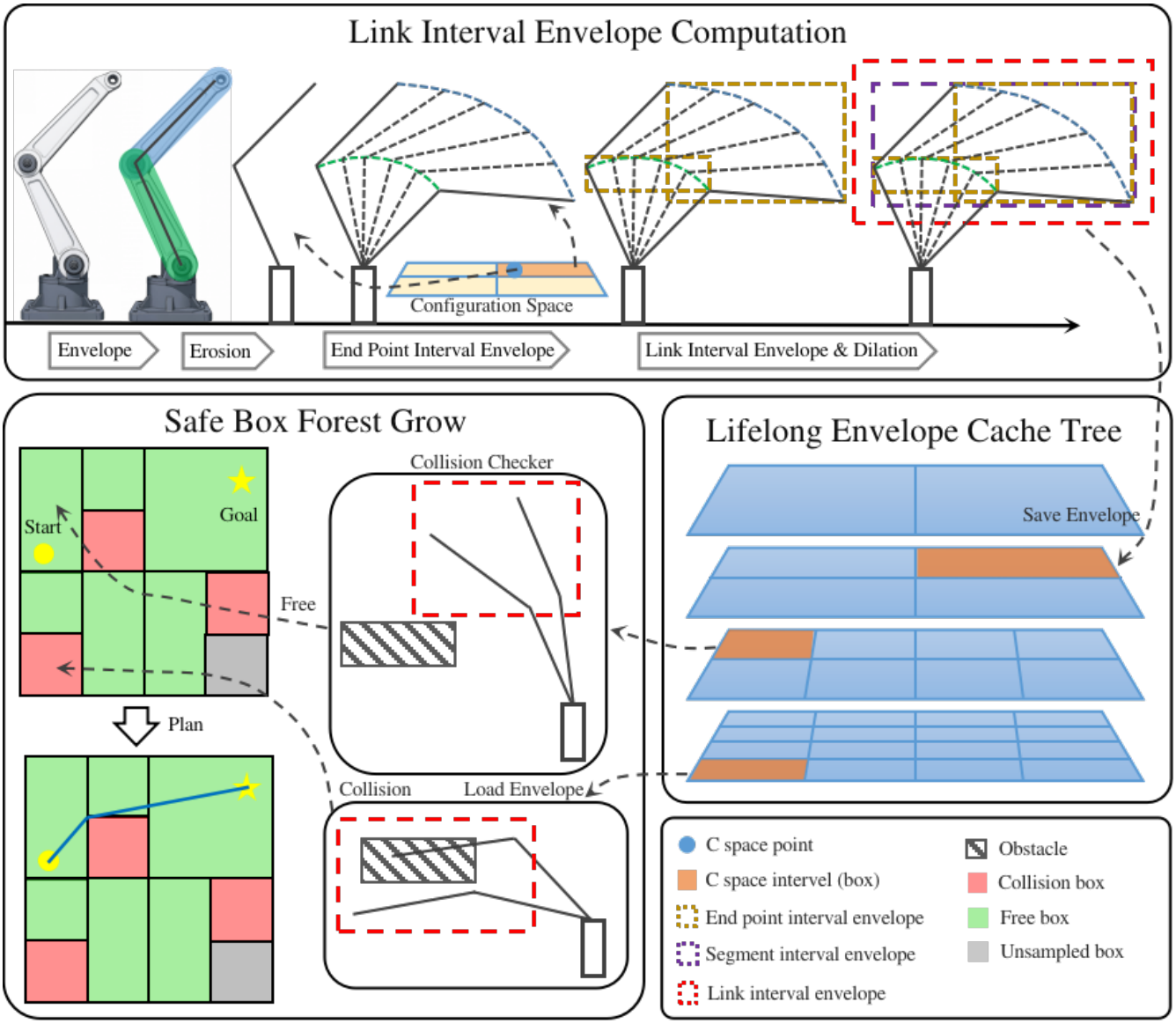


Fig. 1. RBF pipeline for link-envelope-guided multi-query planning. Top: endpoint interval envelopes induce link interval envelopes over a C-space box. Middle: LECT reuses kinematics-keyed endpoint and link-envelope records across repeated builds. Bottom: scene filtering and sample-guided growth expand a connected scene-level box forest for repeated corridor queries.

computer-aided manufacturing (CAM) communities [11], [12]. For a convex body $P = \mathrm{conv}(v_1, \ldots, v_m)$, bounding each generator trajectory by a conservative outer set $\widehat{\Gamma}_i(I)$ yields

$$\mathcal{S}_I(P) \subseteq \mathrm{conv}\Big(\bigcup_i \widehat{\Gamma}_i(I)\Big),$$

a bound that is implicit in oriented bounding box (OBB) tree construction for swept mesh primitives [13].

For capsule links this bound reduces to bounding only the two segment endpoints. The Proximity Query Package (PQP) exploits exactly this Minkowski decomposition ($C = S \oplus B_2(r)$, sweep of C reduces to sweep of S plus radius lift) [14]; Gilbert–Johnson–Keerthi (GJK) and its robust implementation by van den Bergen apply the same identity for distance queries [15], [16]; the Flexible Collision Library (FCL) generalises the swept-sphere interface to a full collision library [17].

C. Interval Forward Kinematics and the CCD Endpoint-Bounding Pipeline

Interval arithmetic [18], [19] provides inclusion-monotone enclosures of real-valued functions over parameter intervals. Snyder applied interval arithmetic to parametric geometry to compute conservative bounding boxes for curves and surfaces without mesh sampling [20]. Merlet extended DH-chain interval propagation to certified workspace analysis for parallel and serial manipulators [21], [22].

These ideas meet directly in the continuous collision detection (CCD) literature for articulated bodies. Redon, Kim, Lin, and Manocha [23] model each robot limb as a line-swept sphere (LSS, i.e. capsule) and propagate interval Denavit-Hartenberg (DH) transforms along the kinematic chain to obtain *interval vectors bounding the two LSS endpoint positions* over a motion time step. Alge-

braically, this is the radius-isolation identity $C = S \oplus B_2(r)$ followed by endpoint bounding of the zero-radius segment S and a final radius lift. The resulting swept capsule is bounded by padding the endpoint bounding box with the link radius. Thus CCD already uses the two-step pipeline

$$\textit{interval FK endpoint bounding} \longrightarrow \text{bbox}([\mathbf{p}_i] \cup [\mathbf{p}_{i+1}]) \oplus B_2(r) = \textit{link swept-capsule bound}$$

which RBF reuses with a different parameter domain and planning role. The journal extension by Redon, Lin, Manocha, and Kim [24] applies the same framework to general robot arms and kinematic trees, making it a representative reference point for articulated-body CCD. Zhang *et al.* [25] replace interval arithmetic with Taylor models in the same pipeline to reduce wrapping errors, analogous to how RBF's HIFK source tightens the unsplit IFK bound by recursively subdividing joint boxes. Schwarzer *et al.* [26] address a multi-query motivation similar to RBF's—reusing collision evidence across planning calls—but at the path-segment level rather than as volumetric C-space records. Corrales *et al.* [27] apply sphere-swept line (SSL) bounding volumes dynamically for human–robot safety monitoring using the same structural observation that bounding a capsule's motion reduces to bounding its two endpoints.

D. Novelty Boundary: Prior CCD vs. RBF

The two-step endpoint-interval-to-swept-capsule pipeline, including radius isolation and final radius lift, is therefore already present in the CCD literature for time-parameterized motion. The contribution boundary in this paper is the use of that pipeline as a C-space free-region-construction primitive. The differences are: (a) the parameter domain is a *C-space joint-box interval* rather than a trajectory time step, so the goal is to validate a volumetric free-space region rather than to test a single path segment; (b) the endpoint source is a switchable certified interface—IFK_AA and HIFK at different subdivision depths—rather than a fixed algorithm, allowing the tightness vs. cost trade-off to be controlled without changing the link-representation or planner layers; (c) compact per-representation link records (AABB and SupportHull) are organized for staged filtering during box growth, a design dimension absent from CCD libraries; (d) accepted boxes are assembled into a typed corridor graph whose overlap edges, tolerance-gap candidates, and segment witnesses have different certificate semantics; and (e) the endpoint and link-envelope records are organized in LECT, a persistent *kinematics-keyed* cache that reuses evidence across multi-query builds in related scenes, a reuse level not addressed by CCD libraries or path-segment caches. This distinction fixes the novelty boundary used throughout the paper. CCD uses the endpoint-bounding pipeline to enclose the swept volume of a body over a given time interval. RBF uses the same geometric pipeline over a joint box to validate candidate volumetric free-space regions, stores compatible evidence for replay, and organizes accepted regions into a queryable planning graph. Link envelopes therefore reduce repeated geometric work during free-region construction, while the query validation remains the acceptance rule for post-processed paths.

E. Multi-Query Reuse and Adaptive Configuration-Space Decomposition

Multi-query planners typically amortize effort by reusing a graph structure such as a roadmap. PRM is the classical example: build a connectivity graph once and reuse it for many start-goal pairs [6], [28]. LazyPRM, lazy edge checking, and certificate-based planners postpone or cache edge validation so that online search does not repeatedly evaluate the same discrete local connections [29], [30]. Experience-based and trajectory-library systems reuse prior solutions or local motion snippets, often as initialization or bias for a new query rather than as a conservative decomposition of a region of configuration space [31].

RBF separates reuse into two levels. Within a fixed scene, the constructed box forest and corridor graph are the reusable planning object, so later queries reuse volumetric boxes rather than only paths, binary collision outcomes, or zero-volume edges. Across related builds or local scene edits, LECT can replay kinematics-keyed envelope evidence while the scene-dependent free-space decision is recomputed. PRM-like methods reuse connectivity and must revalidate affected edges after scene changes. Convex-region planners reuse higher-quality regions, but those regions are usually coupled to the obstacle layout in which they were constructed. RBF makes the boundary explicit: scene-level boxes are reused inside one environment, while kinematic envelope records can outlive that environment.

The method also inherits the central idea of adaptive cell decomposition and subdivision planning: represent free space by cells, split cells where the current evidence is insufficient, and search the adjacency graph induced by the accepted cells [32], [33]. The difference is the operating regime. Classical cell-decomposition examples are often low-dimensional workspace or explicit C-space constructions, whereas RBF applies the same adaptive-cell idea to high-dimensional manipulator C-space through a link-envelope oracle. Scene-specific obstacle filtering is layered on top of reusable kinematics-keyed envelope evidence, with LECT available only as a cache for expensive records and the scene-level forest serving as the direct multi-query cache. The trade-off is explicit: RBF accepts axis-aligned boxes and over-approximation in exchange for simple validation, adjacency maintenance, and repeated-query throughput.

F. Sampling-Based Planning and Path Refinement

Sampling-based planners remain standard baselines when the goal is to solve a single query quickly. RRT-Connect is particularly effective in manipulator planning

because bidirectional growth often succeeds with very little global structure [34]. Asymptotically optimal and informed variants such as RRT*, Batch Informed Trees (BIT*), and Fast Marching Tree (FMT*) improve path quality and search efficiency when more time is available [7], [35], [36], [37]. The Open Motion Planning Library (OMPL) has made these methods easy to deploy and benchmark in a common software stack [38], [39]. However, standard RRT variants produce single-use zero-volume evidence and typically restart from scratch on the next query.

By contrast, RBF uses sample-guided, frontier-biased expansion to materialize C-space boxes from the envelope oracle, producing a volumetric corridor structure that can be queried repeatedly within the same environment. Trajectory optimizers address a different stage of the pipeline. CHOMP, STOMP, TrajOpt, and Gaussian-process motion planners refine an initial path by local descent or sequential convex optimization [40], [41], [42], [43]. RBF is complementary to these methods: it supplies a reusable region graph and leaves local smoothing, repair, and path checking to the downstream query pipeline.

Table I summarizes the resulting design trade-offs. The comparison is qualitative: it identifies what type of object each family reuses and where RBF fits in the chain from geometric evidence to C-space free-region planning, rather than ranking methods by the experimental results reported later.

III. Link Interval Envelopes

This section defines the geometric primitive used by RBF: a reusable map from a joint box to compact link interval envelopes. The goal is not exact swept volume computation, but a low-cost outer representation for box growth. Strict endpoint sources give conservative certificates; tighter sampled or support-based sources act as filters before query validation.

Let $\mathcal{Q}$ be the joint-limit box, $I \subseteq \mathcal{Q}$ a joint interval, and $\mathcal{O} \subset \mathbb{R}^3$ the obstacle set. Endpoint envelope $E_k(I)$ outer-bounds endpoint k, and link envelope $B_\ell(I)$ outer-bounds link ℓ over I, yielding the primitive $I \mapsto \{E_k(I)\} \mapsto B_\ell(I)$. Conservative endpoint evidence gives a conservative link envelope; empirical evidence is used only as a filter.

Assumption 1 (Collision model for certificates)**.** *The formal certificates in this paper concern collision between the configured active rounded links and the current workspace obstacle set $\mathcal{O}$. Joint limits are enforced by $I \subseteq \mathcal{Q}$. Self-collision constraints, attached objects, or task-specific forbidden regions can be included by adding the corresponding link-obstacle or link-link checks to the same scene-validation policy; if they are not enabled, they are outside the certificate stated below.*

A. Geometric Foundation

Let $T(q)x = R(q)x + t(q)$ denote the rigid transform induced by a configuration $q \in I$. For a reference body $C_0 \subset \mathbb{R}^3$, its pose at q is $C(q) = T(q)C_0$. We write $\mathcal{S}_I(C_0)$ for the exact workspace set swept as q ranges over the joint box I and $\mathcal{C}_I(C_0)$ for its convex envelope. RBF usually works with the convex envelope because the downstream representations are AABBs or support-function hull records. The construction below deliberately reuses two classical facts—capsule Minkowski decomposition and convex-hull generator bounding—that already appear in swept-sphere collision libraries and articulated-body CCD work [23], [24]. The change is the domain and downstream object: the parameter set is a C-space joint box, and the output is a candidate box-region for a planning graph.

Radius isolation. For a capsule $C = S \oplus B_2(r)$, rigid motion commutes with the fixed Euclidean ball, giving exact two-stage sweep bounds:

$$\mathcal{S}_I(C) = \mathcal{S}_I(S) \oplus B_2(r), \qquad \mathcal{C}_I(C) = \mathcal{C}_I(S) \oplus B_2(r).$$

RBF therefore follows the same centerline-first, radius-lift construction used by swept-sphere collision methods. This identity is exact for capsules; for sharp polytopes it requires the shape to be a rounded offset body.

Convex-hull generator bound. For a convex body $P = \mathrm{conv}(v_1, \ldots, v_m)$, bounding each vertex trajectory by an outer set $\widehat{\Gamma}_i(I)$ suffices to bound the swept polytope:

$$\mathcal{S}_I(P) \subseteq \mathrm{conv}\Big(\bigcup_i \widehat{\Gamma}_i(I)\Big).$$

For a capsule link the seed is the center segment $[a, b]$, so only two endpoint trajectories need to be bounded.

Combining these two facts yields the practical recipe. For a capsule link $[a, b] \oplus B_2(r)$, RBF bounds only the two endpoint trajectories over the joint box, forms their convex hull, and applies the radius lift:

$$\widehat{\mathcal{C}}_I([a, b] \oplus B_2(r)) = \mathrm{conv}\big(\widehat{\Gamma}_a(I) \cup \widehat{\Gamma}_b(I)\big) \oplus B_2(r).$$

The numerical task reduces to computing reusable envelopes for two endpoint trajectories per link over I. Those records are then used by the planner to classify I as a candidate box-region, not merely to answer a path-segment CCD query.

Proposition 1 (Endpoint envelopes induce link interval envelopes)**.** *Consider a rounded link $L = [a, b] \oplus B_2(r)$ and a joint interval I. If endpoint envelopes $E_a(I)$ and $E_b(I)$ satisfy $\Gamma_a(I) \subseteq E_a(I)$ and $\Gamma_b(I) \subseteq E_b(I)$, then*

$$\mathcal{S}_I(L) \subseteq \mathrm{conv}\big(E_a(I) \cup E_b(I)\big) \oplus B_2(r).$$

Consequently, any outer representation of the right-hand side is a valid link interval envelope for the whole joint box I.

Proof. For each $q \in I$, the center segment of the moved link is the convex hull of the two moved endpoints, so $T(q)[a, b] \subseteq \mathrm{conv}(\Gamma_a(I) \cup \Gamma_b(I))$ over the entire interval. The endpoint inclusions place this set inside $\mathrm{conv}(E_a(I) \cup E_b(I))$. Adding the fixed radius ball commutes with the rigid-motion sweep of the capsule, giving the stated containment. □

Corollary 1 (Conservative joint-box validation)**.** *Let $I \subseteq \mathcal{Q}$ be a joint box and let $\{B_\ell(I)\}$ be conservative link interval envelopes for all active rounded links. If $B_\ell(I) \cap \mathcal{O} = \emptyset$*

for every link ℓ, then every configuration $q \in I$ is collision-free with respect to the collision model in assumption 1.

Proof. By proposition 1, each conservative link envelope contains the swept workspace volume of its link over all configurations in I. Disjointness from $\mathcal{O}$ for every active link therefore excludes workspace obstacle collision for the whole manipulator at every $q \in I$ under assumption 1. □

a) Numerical conservatism.: The certificate above is a mathematical containment statement. A row or mode is claimed to inherit it only when endpoint propagation, envelope padding, and obstacle-disjointness tests are configured as outward bounds under the stated collision model. In practice this requires interval or affine-arithmetic rounding/padding that covers floating-point residuals, conservative treatment of obstacle and link radii, and collision-test tolerances that cannot turn an intersecting envelope into a disjoint one. Empirical endpoint sources, uncovered SupportHull/GJK tolerances, or post-processing outside the conservative box union are therefore treated as filters whose returned paths must pass the independent query-validation path; they are not used to extend corollary 1 and theorem 1.

B. Endpoint Interval Envelopes

Figure 1 illustrates the AABB extraction pipeline. Endpoint interval axis-aligned bounding boxes (iAABBs) are computed by interval forward kinematics (IFK) or tighter certified AA-based variants, combined into a conservative zero-radius centerline envelope, and padded by the link radius before collision querying. The remaining numerical task is to outer-bound each endpoint trajectory over I by a reusable set $E_k(I)$. Endpoint envelopes are the common interface between kinematic propagation and the downstream link-envelope representations.

For endpoint k, let $\Gamma_k(I)$ denote its exact workspace trajectory over interval I. Any conservative set that contains $\Gamma_k(I)$ is an endpoint interval envelope. The basic endpoint record stores this envelope as an axis-aligned bounding box, because boxes are efficient to store, easy to combine, and directly reusable by all downstream link representations.

a) Interval forward kinematics (IFK).: The IFK endpoint source used by the planner is backed by affine-arithmetic forward kinematics. It propagates workspace bounds along the DH chain and provides the lowest-cost certified member of the endpoint-source family. Its limitation is the usual wrapping effect: on wide boxes or long chains, one evaluation accumulates residual over-approximation across the full joint box.

b) Hierarchical IFK (HIFK).: HIFK further reduces wrapping by recursively bisecting the joint box and evaluating AA-FK at the leaves before hulling the child endpoint boxes. HIFK is therefore a distinct endpoint source rather than a naming variant of unsplit IFK: a depth-1 HIFK evaluation already splits once and hulls two leaves, whereas IFK performs one unsplit AA-FK pass on the original box. The split depth is an endpoint-source parameter: larger depths reduce wrapping at the cost of additional FK evaluations.

c) Incremental FK state reuse.: Tree descent also carries a transient FK state: interval-valued prefix transforms $[\boldsymbol{T}^{0:0}], \ldots, [\boldsymbol{T}^{0:n}]$ and the per-joint DH interval matrices used to build them. The full prefix chain is computed once at the root or after a restart; when only I_k changes, prefixes before k remain valid and only the suffix is recomputed. This short-lived state is not the persistent cache itself; it only keeps endpoint materialization efficient before LECT decides whether to retain the evidence.

Taken together, these sources define a clear certified spectrum: IFK_AA is the lowest-cost source, while increasing HIFK depth offers progressively tighter outer bounds at increasing cost. Endpoint envelopes are the right intermediate abstraction for the planner because they isolate the kinematic bounding problem from the scene-query and graph-building problems. The downstream link-envelope layer remains fixed while the upstream source changes with the certification and runtime budget, and LECT can cache the resulting evidence without depending on the obstacle set.

C. Link-Envelope Representations

Once endpoint envelopes are available, the remaining design question is how to represent the swept link volume compactly for box growth, collision filtering, and optional reuse. For a capsule link, RBF does not reason about the thick link directly. It first bounds the zero-radius center segment between the two endpoint envelopes and then pads that centerline envelope by the link radius. A conservative link interval envelope $B_\ell(I)$ is any set that contains the full swept capsule over interval I. In the box-based realization used below this is written as $B_\ell(I) = B^\circ_\ell(I) \oplus \mathcal{R}(r_\ell)$, where $B^\circ_\ell(I)$ bounds the centerline sweep and the padding operator $\mathcal{R}$ becomes B_∞ for AABB envelopes.

The following representations share the same endpoint evidence but preserve different amounts of downstream geometric detail.

a) AABB envelope.: AABB envelopes are the simplest option. The standard LinkIAABB construction takes the two endpoint envelopes, wraps them in one box, and pads that box by the link radius:

$$B_\ell(I) = \mathrm{bbox}\big(E_{\ell-1}(I) \cup E_\ell(I)\big) \oplus B_\infty(r_\ell).$$

This representation can be tightened by subdividing the reference center segment into S pieces, bounding each piece separately, and retaining the union of the padded sub-envelopes. Larger S preserves more directional variation along the link while keeping reconstruction efficient once endpoint envelopes are available. The main caveat is that the subdivision must remain explicit: if all sub-envelopes are collapsed back into one global AABB, much of the gain disappears.

b) SupportHull envelope.: When the endpoint envelopes are convex, their convex hull preserves directional structure that a single AABB discards. SupportHull is the compact runtime realization of that idea: instead of storing an explicit dense hull, it keeps the two endpoint AABBs and the link radius as a support-function record. For a query direction d, the support point is the better of the proximal and distal AABB support points, and collision refinement against an obstacle AABB uses a fixed-size Gilbert–Johnson–Keerthi (GJK) narrow phase with the obstacle inflated by the link radius and safety margin. This retains much of the convex- hull directional fidelity while keeping the record compact.

Thus the envelope layer is modular with respect to the upstream kinematics solver: endpoint records determine the link envelope, and the chosen representation sets the planning-time cost/tightness trade-off.

IV. Lifelong Envelope Cache Tree

The planner can materialize envelopes online; LECT is an optional replay layer for repeated builds or expensive envelope representations. It stores scene-independent endpoint/link-envelope evidence keyed by robot kinematics and joint intervals. Obstacle tests, box labels, graph connectivity, and query outcomes remain scene-local.

LECT is therefore separate from the scene-level RBF cache. The forest and corridor graph answer multiple queries in one fixed environment; LECT sits below that cache and reuses compatible kinematic evidence during builds, rebuilds, or local refills.

A. Cache Key and Split Policy

LECT is a dynamically grown binary KD tree over the joint-limit box $\mathcal{Q}$. Each node represents one joint interval and stores interval bounds, split metadata, cached endpoint evidence from which link envelopes can be reconstructed, and residency state for the runtime. The key property is that the stored evidence is *kinematics-keyed* rather than *scene-keyed*: later scenes may replay a materialized envelope record, but they must rerun their own scene-local obstacle tests. Nodes use a heap-style breadth-first indexing convention (0 at the root and $2p+1, 2p+2$ for children), which gives a finite indexed tree with configurable maximum depth $D_{\max}$.

a) Evidence/label separation.: An envelope record is replay-compatible only when the robot model, endpoint source, envelope representation, split descriptor, link radii, and active-link set match the current build. Replaying such a record imports endpoint and link-envelope evidence, not a free-space label. Every scene must still test the reconstructed link envelopes against its own obstacle set before a box can be accepted into the forest. This separation is what allows LECT to accelerate repeated geometry materialization without becoming a stale collision database.

The split policy is configurable and canonical for a given cache descriptor. The implemented schedules include round-robin, widest-root, and AAFK-volume-min. A build fixes one descriptor so that replay compatibility is well-defined; alternative descriptors are treated as separate cache settings. For a parent interval I, let I_d^- and I_d^+ be the two children obtained by splitting joint dimension d at its midpoint, and let $\mathcal{V}(I) := \sum_\ell \mathrm{vol}(B_\ell(I))$ be the cumulative downstream envelope volume. The AAFK-volume-min schedule chooses depth-wise split dimensions that minimize a worst-child envelope burden,

$$d^* = \arg\min_d \max\left\{ \sum_\ell \mathrm{vol}(B_\ell(I_d^-)), \sum_\ell \mathrm{vol}(B_\ell(I_d^+)) \right\}.$$

The resulting descriptor is depth-synchronous: all nodes at the same depth use the same scheduled dimension when that dimension is still wide enough, otherwise the runtime falls back to a valid widest dimension. Candidate dimensions may also be filtered by FK reachability, since splitting a joint beyond the last active frame that contributes to any link endpoint cannot tighten the endpoint envelopes.

b) Sparse interval staging.: The compressed runtime path does not require every intermediate heap node on a deep split path to be materialized. A materialized planning cell can instead be represented by its root copy, per-dimension target-grid interval $[\iota_d^-, \iota_d^+)$, split-count vector, and exact joint interval. This is a Patricia-style overlay on the depth-synchronous split policy: branching and terminal cells are indexed, while unvisited ancestors remain virtual. The staging record also carries the interval fingerprint and split-policy hash used by LECT's exact interval replay path. Thus evidence replay remains keyed by the robot-compatible joint interval through `endpoint_for_box_exact`; if no exact record is available, the same interval is live-materialized and then validated against the scene. Scene labels are still stored only in the forest or partition layer, not in LECT.

B. Runtime Use and Validation Boundary

LECT is queried by FINDFREEBOX, not by the graph search itself. Given a joint interval, the cache either replays endpoint evidence or materializes it online, then derives the requested link representation. Directional records are used as refinement evidence rather than as standalone free-space certificates: the runtime first evaluates padded link AABBs, then uses SupportHull/GJK only for survivors of the AABB test. When a node is split, child envelopes may refine the parent witness, but any compact parent record remains representation-dependent and is rechecked against the current obstacle set before a box can enter the forest.

C. Storage and Warm-Build Semantics

Persistence is selective. Endpoint evidence is the common reusable record, but simple IFK+AABB evidence can be lower-cost to recompute online than to reload. SupportHull records are stored only when their tighter filters are expected to amortize storage and lookup cost. Reuse accounting should therefore attribute cache hits

only to compatible kinematic evidence replay and not rely on additional symmetry-transfer assumptions.

The prewarm phase follows the same separation. It materializes FK/link-envelope evidence only on the target leaf layer and reconstructs upper layers by child-hull propagation during checkpoint. This avoids charging direct FK at multiple depths for the same cache and keeps parent witnesses tied to the leaf records that generated them. Exact leaf evidence and propagated child-hull evidence are both treated as envelope records by the replay path; provenance does not create a separate reuse or validation mode.

In the warm-build path, a cache hit reconstructs the requested link-envelope representation from stored endpoint records and revalidates it against the current obstacles. Warm builds still regrow scene-specific boxes; persisted box labels are not imported as certificates. Obstacle checking, adjacency, scene-level query reuse, and reactions to changing $\mathcal{O}$ remain in the forest routines. Fixed-replay studies can disable writeback, while cache-growth studies use a separate writeback policy.

V. RapidBoxForest Construction

This section describes the RBF planning layer. The algorithm grows a scene-specific forest of C-space boxes and exports it as a typed corridor graph for repeated queries. This forest-and-graph pair is the scene-level cache: while the obstacle set is unchanged, new queries use membership lookup and graph search instead of regrowing the forest. Samples choose frontiers, the envelope oracle validates boxes, and accepted boxes extend the connected region graph. The link interval envelopes of section III and LECT of section IV provide the kinematic evidence used by this scene-level decomposition.

a) Box, cell, and domain terminology.: In the planner, a *box* is an axis-aligned joint interval $B = \prod_d [l^{(d)}, u^{(d)}] \subseteq \mathcal{Q}$. The term *cell* is used when viewing the same object as an adaptive-decomposition element. A *collision domain* is a cell that has not been admitted to the forest under the scene-validation policy but is retained as a bounded refinement or update region. Thus boxes, cells, and domains share the same interval representation; the terms distinguish their role in the construction.

b) Validation terminology.: Throughout this section, a *conservatively validated box* is a joint box whose conservative link envelopes are disjoint from the current obstacle set. These boxes support the formal corridor statement below. A *performance-mode box* is accepted by a tighter nonconservative filter and is therefore treated as a planning candidate until the recovered path passes query validation. We use *validated box* for the box admitted by the selected scene-validation policy. This policy is the box-admission rule used during forest construction: it may be the conservative envelope-disjointness test, a performance-mode filter, or the same test augmented with task-specific scene constraints. It is separate from the query-validation path, which checks recovered paths that do not inherit the conservative box-corridor certificate. We explicitly state when the stronger conservative certificate is required. Paths that leave the conservative box union through repair, smoothing, or external composition are charged where used and passed through the same query validation.

TABLE II
Terminology used by the RBF construction and evaluation.

Term	Meaning in this paper
Box	Axis-aligned joint interval in $\mathcal{Q}$.
Cell	The same interval viewed as an adaptive-decomposition element.
Domain	Bounded cell retained as a refinement or update region.
Collision domain	Non-admitted domain with blocker metadata.
Certified-free cell	Terminal cell proven free by conservative scene validation.
Terminal cell	Leaf-sweep cell no longer split under the current budget/policy.
Conservatively validated box	Box whose conservative link envelopes are disjoint from $\mathcal{O}$.
Performance-mode box	Box admitted by a tighter filter; needs query validation.
Validated box	Box admitted by the selected scene-validation policy.
Scene-validation policy	Box-admission rule used while building or updating the forest.
Query-validation path	Independent path check used when the conservative corridor certificate is not inherited.

The build process has four stages. First, a seed-guided local oracle adaptively refines and classifies a C-space cell around a selected configuration. Second, the leaf-sweep mode constructs a shallow, reusable partition of the current scene and identifies conservative refinement domains. Third, repeated frontier expansion or restricted refinement grows validated cells into a connected forest, giving the method a region-valued, frontier-biased construction. Fourth, consolidation coarsens redundant boxes before the structure is exported as a query graph. The consolidation pass is part of the reusable build phase.

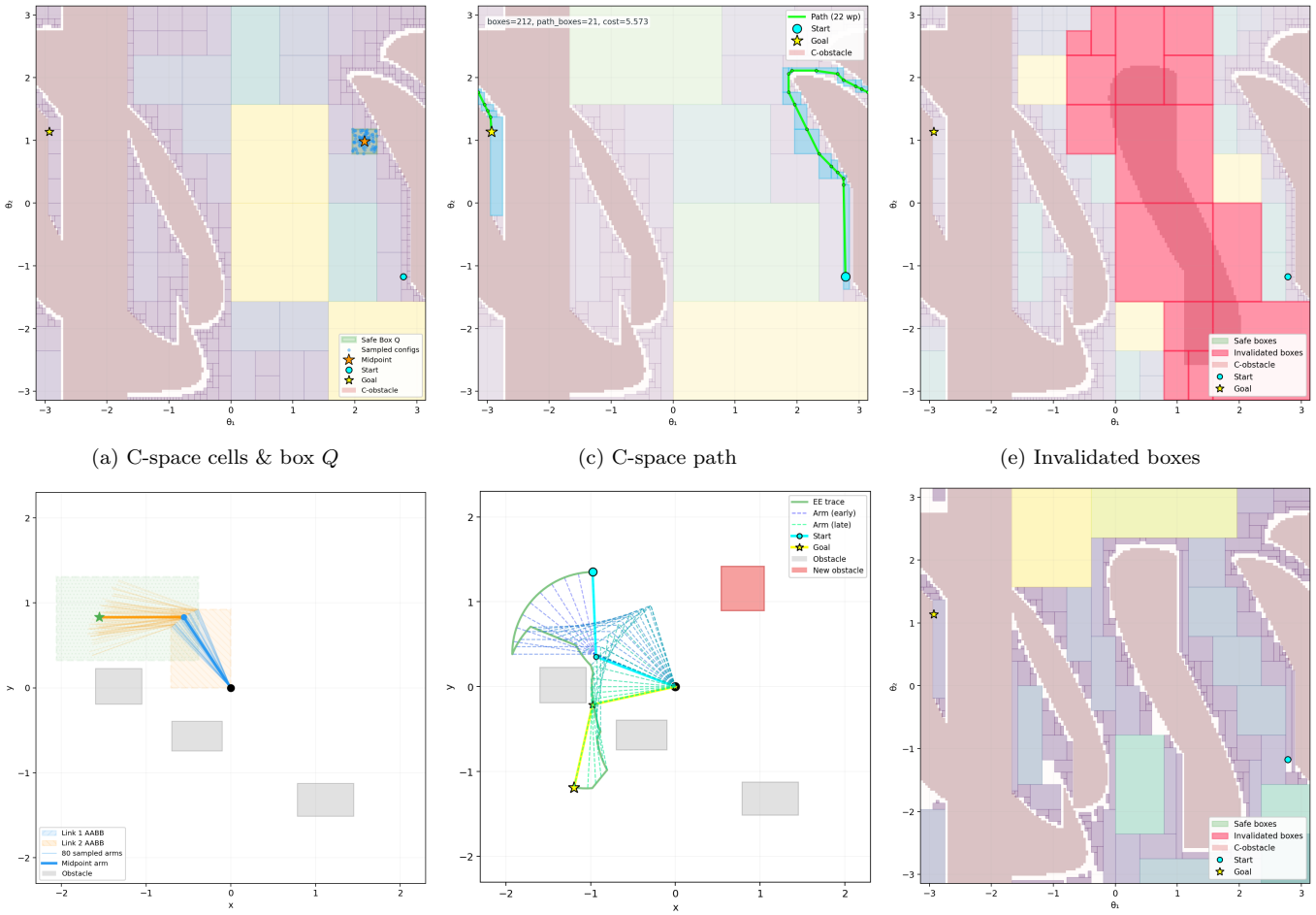
A. Construction Objective

Let $\mathcal{Q}$ denote the joint-limit box and let $\mathcal{O}$ denote the current obstacle set. The construction objective is to build a finite family

$$\mathcal{F} = \{B_1, \ldots, B_N\}, \qquad B_i \subseteq \mathcal{Q},$$

such that every B_i is admitted by the selected scene-validation policy, the union $\bigcup_i B_i$ covers as much query-relevant free space as possible under the available box, time, and depth budgets, and the induced adjacency graph supports efficient downstream search. In conservative mode these admitted boxes are formal free-space certificates; in performance-oriented modes they are planning regions whose recovered paths require query validation. This object is scene-specific: changing $\mathcal{O}$ changes which intervals are accepted into the forest, even though the underlying envelope evidence in LECT remains reusable.

The central structural rule is that the forest is not merely a free-space cover, but a *corridor-compatible* free-



(a) C-space cells & box Q (c) C-space path (e) Invalidated boxes

(b) Workspace AABBs of Q (d) Workspace path (f) After regrowth

Fig. 2. Illustrative RBF build, query, and obstacle-update cycle on a planar 2-link manipulator. (a, b) Adaptive C-space cells and workspace AABB envelopes of the highlighted box Q. (c, d) Box-corridor path in C-space and its workspace trajectory. (e, f) After obstacle insertion, intersecting boxes are invalidated (red), and localized refill plus graph repair updates the explicit forest without requiring a full rebuild.

space cover. Boxes are inserted under a connected-box constraint.

Viewed as adaptive cell decomposition, this objective is intentionally workload-biased rather than globally exhaustive. The planner does not try to classify every cell in $\mathcal{Q}$. It refines and accepts cells near query-relevant frontiers, task seeds, and connector candidates, then keeps the resulting adjacency graph as the reusable planning object.

a) Connected-box rule.: For boxes

$$B_i = \prod_{d=1}^{n} [l_i^{(d)}, u_i^{(d)}], \qquad B_j = \prod_{d=1}^{n} [l_j^{(d)}, u_j^{(d)}],$$

RBF treats them as connected when, in every coordinate, the two intervals either overlap or are separated by at most a small tolerance ε_c:

$$\max\{l_i^{(d)}, l_j^{(d)}\} \le \min\{u_i^{(d)}, u_j^{(d)}\} + \varepsilon_c, \qquad d = 1, \dots, n.$$

When $\varepsilon_c = 0$, this reduces to standard interval intersection or boundary touching in every dimension. A small positive tolerance absorbs the boundary offset used by grower seeds and floating-point roundoff.

This rule is deliberately weaker than requiring a full shared face. It treats overlap, touching, and tolerance-scale gaps uniformly as graph-connectivity evidence, while validity remains attached to the accepted box union itself. Only overlapping or touching box chains directly imply a contained corridor; tolerance-gap edges are query candidates that must be bridged by contained waypoints or by explicit segment validation. The output of the construction stage is therefore a forest of validated boxes whose graph connectivity is aligned with the corridor object queried later, without overstating a stricter face-sharing invariant than the grower actually maintains.

B. Find Free Box

FINDFREEBOX descends LECT from the root toward the leaf containing seed $q \in \mathcal{Q}$, materializing endpoint and link envelopes lazily along the path, honoring reserved-node and skip-to-depth controls, and returning the first validated ancestor on the seed path. This is the largest canonical node on that path accepted by the scene-validation policy before further descent is needed. Tree topology and endpoint records are reused from the scene-independent cache, while scene-specific validation is performed on each grower visit. Algorithm 1 gives the pseudocode.

The termination behavior follows directly from the depth cap. Under $D_{\max}$, each loop iteration either returns

Algorithm 1 FindFreeBox: local box materialization around a seed.

Input: Seed configuration q, LECT T, obstacle set $\mathcal{O}$, depth cap $D_{\max}$, skip depth D_{skip}
Output: Validated interval I_v or failure

$v \leftarrow \text{root}(T)$; $d \leftarrow 0$; initialize current interval and FK state
while $d \le D_{\max}$ **do**
  Cache materialization
  materialize the endpoint and link envelopes of v if needed
  Reservation and scene validation
  if v is reserved and cannot be split further **then**
    return failure
  end if
  if $d \ge D_{\text{skip}}$ and v passes the scene-validation policy **then**
    return interval I_v
  end if
  Refinement
  if $d = D_{\max}$ **then**
    return failure
  end if
  if v has no children **then**
    split v along the depth-synchronous split dimension of depth d
  end if
  $k \leftarrow$ split dimension of v
  $v \leftarrow$ the child of v whose interval contains q; update the current interval
  update the FK state from dimension k onward; $d \leftarrow d + 1$
end while
return failure

immediately or descends one more level in the tree, so the procedure can make at most $D_{\max} + 1$ decisions. In strict conservative mode, any returned interval has passed envelope disjointness against $\mathcal{O}$; in performance-oriented modes, the returned interval is a validated planning region that remains subject to the common query-validation path.

C. Adaptive Leaf Sweep and Restricted Refinement

The fast construction path uses FINDFREEBOX inside an adaptive terminal cell routine. Rather than relying only on stochastic frontier proposals to discover query-relevant coarse regions, RBF starts from a shallow depth-synchronous cell set and recursively classifies only cells that remain useful under the build budget. A terminal cell is one of five types: certified-free, certified-occupied, inconclusive-domain, deferred, or covered by an already accepted box. A certified-free cell is inserted into the initial forest and its descendants are pruned. An inconclusive cell is split only when its depth, blocker report, adjacency to accepted boxes, and seed evidence suggest that further refinement can improve the reusable cover. The fixed virtual leaf sweep is recovered as a special case that forces every inconclusive cell to split until the terminal depth.

During adaptive classification, RBF also records a blocker set

$$\beta(C) \subseteq \{1, \dots, |\mathcal{O}|\}$$

for each non-free or deferred collision domain. This set is conservative update metadata: it contains obstacle groups that were observed as blockers or were not proven free by the grouped shallow sweep. The blocker set is an over-approximate update explanation; certification of boxes entering the forest remains governed by the standard envelope validation rule.

An optional occupied predicate is used only when it can prove the whole cell is inside an obstacle. Otherwise the cell remains inconclusive or deferred.

Lemma 1 (Signed-distance occupied pruning)**.** *Let x_0 be a material point on link ℓ, let q_c be the center of cell I, and let ϕ_O be the signed distance to obstacle O in workspace. If the workspace motion of this material point over I is bounded by $\rho_{\ell,x_0}(I)$ and*

$$\phi_O(T_\ell(q_c)x_0) + \rho_{\ell,x_0}(I) + \epsilon_{\text{num}} < 0,$$

then every configuration in I places that material point inside O, so I can be recorded as a certified occupied domain for pruning.

When such a witness is unavailable, RBF does not infer occupancy; it keeps the cell as an inconclusive or deferred refinement domain.

After the shallow sweep, restricted refinement is performed only inside selected collision domains. Domains are ranked by their adjacency to already certified boxes, volume, and proximity to query-priority configurations. Candidate seeds are placed at interfaces between a collision domain and neighboring free boxes, with additional center and boundary seeds to avoid depending on a single contact point. For a domain C, every accepted refined box B must satisfy

$$B \subseteq C$$

and must be adjacent either to an existing forest box or to a box already accepted from the same domain. Thus the refinement stage can fill narrow passages revealed inside a shallow collision domain without spending its budget on unrelated regions of $\mathcal{Q}$. The connector stage may subsequently add collision-checked segment witnesses between remaining islands, but these witnesses are recorded separately from box-overlap edges.

If N_{mat} cells are materialized or replayed and N_{term} terminal cells are retained, the sweep work is $O(N_{\text{mat}}(C_{\text{env}} + C_{\text{obs}}) + C_{\text{adj}})$ and the terminal storage is $O(N_{\text{term}})$, where C_{env} is the envelope materialization or replay cost, C_{obs} is scene validation cost, and C_{adj} is the indexed adjacency update cost. The important distinction from a fixed virtual layer is that N_{mat} is driven by accepted and frontier cells, not by the full cardinality of $\mathcal{L}_{d_0:d_1}$.

D. Forest Growth

Starting from an initial seed set, the forest is grown by repeatedly invoking the find-free-box procedure on carefully chosen candidate configurations. Each seed first produces a root box through the find-free-box procedure, and these root boxes initialize one or more connected components of the forest. The main grower is an evidence-guided, frontier-biased expansion process rather than a generic random cover. Each iteration chooses a frontier component, proposes a nearby interval worth testing, and accepts the result only when it extends the forest by an admissible connected corridor segment. This keeps the

Algorithm 2 AdaptiveLeafSweepRefine: terminal-cell coverage with restricted refinement.

Input: LECT T, obstacle set $\mathcal{O}$, sweep depths d_0, d_1, refinement budget N_r
Output: Validated forest $\mathcal{F}$ and collision-domain cache $\mathcal{C}$

$\mathcal{F} \leftarrow \emptyset$; $\mathcal{C} \leftarrow \emptyset$
initialize a priority frontier with start cells at depth d_0
while frontier nonempty and build budget remains **do**
  pop the highest-priority cell C
  if C is covered by an accepted box **then**
    mark C covered and continue
  end if
  materialize or replay LECT evidence for C and run scene validation
  if C is certified free **then**
    insert C into $\mathcal{F}$ and prune descendants
  else if lemma 1 certifies C occupied **then**
    record C as a certified occupied terminal domain
  else if depth$(C) = d_1$ or C is low-priority under nogood/connectivity rules **then**
    record C and its blocker set $\beta(C)$ in $\mathcal{C}$
  else
    split C by the configured split policy and push selected children
  end if
end while
build adjacency among boxes in $\mathcal{F}$
for all collision domains $C \in \mathcal{C}$ in priority order **do**
  generate interface, center, and boundary seeds inside C
  for all seed q while budget remains **do**
    $B \leftarrow$ FindFreeBoxInDomain$(q, T, \mathcal{O}, C)$
    if $B \neq \emptyset$, $B \subseteq C$, and B is adjacent to $\mathcal{F}$ or to a prior box from C **then**
      insert B into $\mathcal{F}$ and update local adjacency
    end if
  end for
end for
return $\mathcal{F}, \mathcal{C}$

build process focused on boxes that can improve query connectivity rather than on uniformly covering all of $\mathcal{Q}$.

Each iteration makes two explicit heuristic decisions: it samples a target configuration q_t, then chooses a boundary seed on an existing box face that is closest to that target. The target sampler is a configurable categorical mixture over intertree roots, unexplored-volume targets, query roots, fixed anchors, and uniform roots. Build connectivity is handled primarily by overlap-compatible box insertion; segment bridges remain a secondary mechanism recorded separately from box-overlap edges. The mixture weights, depth caps, cache depth, thread count, and anchor set are planner settings rather than envelope-theory parameters.

Given a target q_t and an existing box $B = \prod_j [\ell_j, u_j]$, the grower enumerates feasible faces $f = (B, d, s)$, where d is a joint dimension and $s \in \{-1, +1\}$ is the lower or upper side. Let τ be the adjacency tolerance and $\epsilon_b = \max(\epsilon_{\text{guard}}, \tau/4)$. A face is feasible only if stepping outside it by ϵ_b remains inside the global root interval. The seed associated with the face is

$$q_f[j] = \begin{cases} u_d + \epsilon_b, & j = d,\ s = +1, \\ \ell_d - \epsilon_b, & j = d,\ s = -1, \\ \text{clip}(q_t[j], \ell_j + \epsilon_b, u_j - \epsilon_b), & j \neq d, \end{cases} \tag{1}$$

with clipping to $[\ell_j, u_j]$ when a box is thinner than $2\epsilon_b$. Faces are ranked by the squared target distance

$$\sigma(f; q_t) = \|q_t - q_f\|_2^2, \tag{2}$$

Algorithm 3 GrowForest: budgeted expansion of a connected box-region forest.

Input: LECT T, obstacle set $\mathcal{O}$, seed set $\mathcal{S}$
Budgets/settings: $N_{\max}, t_{\max}, K_{\text{face}}, K_{\text{fail}}, H_{\text{cool}}$
Output: Validated box-region forest $\mathcal{F}$

$\mathcal{F} \leftarrow \emptyset$; initialize connected components from $\mathcal{S}$
for all $q \in \mathcal{S}$ **do**
  $B \leftarrow$ FindFreeBox$(q, T, \mathcal{O})$
  if $B \neq \emptyset$ **then**
    insert B into $\mathcal{F}$ as a root box
  end if
end for
while $|\mathcal{F}| < N_{\max}$ and time budget not exhausted **do**
  draw target q_t from the configured connector/query/unexplored/uniform mixture
  enumerate feasible faces $f = (B, d, s)$ of frontier boxes and compute q_f and $\sigma(f; q_t)$
  rank the K_{face} lowest-score face seeds
  choose the first ranked seed that is not deferred and not in the frontier-seed cache
  set $(q_{\text{seed}}, B_{\text{near}}) \leftarrow (q_f, B)$ for that face
  $B_{\text{new}} \leftarrow$ FindFreeBox$(q_{\text{seed}}, T, \mathcal{O})$
  if $B_{\text{new}} \neq \emptyset$ and BoxesConnected$(B_{\text{new}}, B_{\text{near}}, \tau)$ **then**
    insert B_{new} into $\mathcal{F}$; UpdateComponents$(\mathcal{F})$
  else
    record the failed LECT leaf; defer it after K_{fail} failures for horizon H_{cool}
  end if
end while
return $\mathcal{F}$

and the grower retains a bounded candidate set of size K_{face} across the scanned frontier boxes. It then tries them in increasing score order, skipping a candidate if the corresponding LECT leaf is temporarily deferred or if a component-connection cache has already recorded the same boundary seed. The first remaining candidate becomes the seed for FINDFREEBOX.

The candidate box is accepted only if FINDFREEBOX validates it and the new box is connected to its parent under the same adjacency predicate used by the query graph: all dimensions overlap up to tolerance τ, and at least one dimension touches within tolerance or all dimensions overlap. Failures are also handled explicitly. When the LECT leaf containing a failed seed reaches the configured hard-frontier failure threshold K_{fail}, the leaf is skipped for a deferral horizon H_{cool} of subsequently accepted boxes, unless a later depth stage raises the active search depth and enables a retry. This deferral rule and the frontier-seed cache do not alter the validation boundary; they only reduce repeated calls on already failed or already covered frontier regions. The resulting method is summarized in algorithm 3.

The same construction also supports coordinated multi-goal growth. Multiple seed configurations can initialize roots in one authoritative forest, allowing the scene-level RBF cache to serve a batch of related queries while sharing LECT evidence across build workers and cache-warm reruns. Staged wavefront growth and task-batched growth are scheduling choices; they do not change the mathematical object produced by the planner, namely the query-ready box-region graph.

Algorithm 4 ConsolidateForest: graph-preserving merging and pruning.

Input: Validated forest $\mathcal{F}$, LECT T, obstacle set $\mathcal{O}$
Output: Consolidated validated forest $\mathcal{F}'$
$\mathcal{F}' \leftarrow \mathcal{F}$
Stage 1: validated connected/contact merging
merge exact connected pairs whose union is itself a single box
for all connected candidate pairs (B_i, B_j) in $\mathcal{F}'$ **do**
  build a candidate parent region H and score its volume expansion
  if the chosen envelope representation of H passes the scene-validation policy **then**
    replace B_i, B_j by H
  end if
end for
Stage 2: containment pruning
remove only contained boxes whose typed adjacency and segment witnesses transfer to selected covering regions
return $\mathcal{F}'$

E. Forest Consolidation

Raw growth tends to produce a forest that is finer than necessary. Consolidation reduces this representation without changing the validation rule or the typed-edge semantics used by corridor search. The planner uses two conservative operations. First, connected pairs whose union is again a box may be merged after validation. Second, more general connected candidate pairs are ranked by a volume-expansion score and accepted only if the candidate hull passes the same scene oracle. A final containment-pruning pass removes boxes that are fully covered by accepted validated regions when their graph incidence can be transferred. The consolidation logic is summarized in Algorithm 4.

Validity follows directly: every accepted merge has itself passed the scene oracle. Containment pruning removes only boxes fully covered by a selected region that inherits their typed adjacency and segment witnesses, preserving the connectivity semantics used by query search.

F. Corridor Query

After growth and consolidation, the forest is exported as a typed graph $G = (V, E_\cap \cup E_\varepsilon \cup E_s \cup E_\pi)$ with one vertex per validated box. Edges in $E_\cap$ connect boxes that overlap or touch in all dimensions; these edges directly preserve a contained box corridor. Edges in E_ε come only from tolerance-scale connected-box gaps and are treated as query candidates rather than as corridor certificates. Edges in E_s are explicit collision-checked segment witnesses. Edges in E_π are conservative portal edges: each stores a compressed edge between two global boxes together with a finite hidden chain of conservatively validated internal boxes. The chain is not materialized as global graph vertices, but it is expanded when the edge is used by query path extraction. Because typed connectivity is maintained during growth and consolidation, online querying reduces to membership lookup and typed graph search rather than post-hoc graph reconstruction.

If disconnected adjacency islands remain after growth, an optional connector stage attempts targeted bidirectional sampling bridges between nearest frontier boxes of different components. A successful bridge is stored as a segment edge rather than as a claim that the two boxes overlap. This distinction is important for both the RBF query layer and the GCS negative controls: a segment edge is a collision-checked path witness between two boxes, whereas a GCS edge over convex regions requires feasible overlap of the regions themselves.

TABLE III
Typed corridor-edge semantics. The theorem column refers to theorem 1.

Edge	Construction condition	Cert.?	Query validation?
$E_\cap$	Boxes overlap or touch	Yes	Only after noncontained edits
E_ε	Tolerance-scale gap	No	Yes
E_s	Collision-checked segment witness	Segment only	Yes for returned path
E_π	Hidden conservative box chain	Yes after expansion	No if fully expanded

This pair $(\mathcal{F}, G)$ is the planning object produced by the build stage. It is also the scene-level RBF cache for subsequent queries in the same environment. Given start and goal configurations q_s, q_g, the planner associates them with start-side and goal-side boxes in $\mathcal{F}$ and then searches G for a connecting box sequence. The online query problem is reduced to box-corridor retrieval inside an already constructed box-region representation.

A shortest path on G yields a candidate box sequence between the two query endpoints. When the recovered geometric path stays inside the union of conservatively validated boxes, it is certified collision-free by theorem 1. Paths that use E_ε or E_s edges, performance-mode boxes, or post-processing steps that leave the conservative box union are treated through the configured query-validation path, and the query interface records these cases through the `QueryResult` counters. The box-corridor statement below applies to paths contained in that union. Portal edges in E_π are nonsegment edges: they inherit the conservative box-corridor certificate only through their stored internal box chain.

For an $E_\cap$-only graph path, this containment condition has a direct constructive interpretation. Each vertex box is convex, and each $E_\cap$ edge means that consecutive boxes overlap or touch. Therefore a continuous piecewise-linear curve can be chosen by selecting one point in each nonempty pairwise intersection and connecting these points inside the corresponding boxes. If q_s and q_g lie in the first and last boxes, respectively, the same convexity argument connects them to the first and last intersection points. The resulting curve is contained in the union of the boxes on the graph path. This is a containment certificate, not a positive-clearance certificate: touching boxes may meet at a lower-dimensional set, and clearance depends on the underlying envelope-obstacle separation or on the configured query-validation test. This construction does not apply to E_ε or E_s edges, whose semantics are candidate gap closure and explicit segment witnessing, respectively.

Corollary 2 (Conservative portal expansion)**.** *Let $(B_u, B_v) \in E_\pi$ store an internal chain $B_u, B_1, \ldots, B_m, B_v$ such that every internal box is conservatively validated and every consecutive pair overlaps or touches. Replacing the portal edge by this chain recovers an $E_\cap$-only box-corridor*

segment. Therefore any query path using such portal edges is covered by theorem 1 after all portals are expanded.

Theorem 1 (Conditional conservative box-corridor path)**.** *Let $\mathcal{F}$ be a forest whose boxes have each passed conservative link-envelope validation against obstacle set $\mathcal{O}$. If a continuous path $\gamma : [0,1] \to \mathcal{Q}$ satisfies $\gamma([0,1]) \subseteq \bigcup_{B\in\mathcal{F}} B$, then every configuration on γ is collision-free with respect to the collision model in assumption 1.*

Proof. For any $s \in [0,1]$, the containment assumption gives a conservatively validated box $B \in \mathcal{F}$ with $\gamma(s) \in B$. By construction, validation of B means every conservative link envelope for B is disjoint from $\mathcal{O}$. Because each such envelope contains the link geometry for every configuration inside B (corollary 1), the robot geometry at $\gamma(s)$ is also disjoint from $\mathcal{O}$. Since s was arbitrary, the whole path is collision-free. □

Proposition 2 (Resolution-limit sufficient conditions)**.** *Assume that the obstacle set is closed, a reference path $\gamma : [0,1] \to \mathcal{Q}$ has positive clearance $\delta > 0$ under the collision model of assumption 1, joint boxes are exhaustively refined with diameter tending to zero, and the conservative endpoint/link-envelope over-approximation converges to the exact link sweep as box diameter tends to zero. Then there exists a finite chain of sufficiently small conservatively validated boxes whose union contains $\gamma([0,1])$.*

Proof. Positive clearance and closed obstacles imply an open collision-free tube around the compact set $\gamma([0,1])$. By envelope convergence, every point on the path has a small enough joint-box neighborhood whose conservative link envelopes remain inside that tube. These neighborhoods form an open cover of the compact path image, so a finite subcover exists. Exhaustive refinement with vanishing box diameter eventually generates boxes contained in that subcover; ordering them along the continuous path gives the required finite chain. □

a) Reporting protocol.: Theorem 1 is a conditional statement: it covers path portions contained in a conservatively validated box union. Post-processing inherits this certificate only when it preserves that containment condition. The result is therefore a *soundness* statement for the returned box-contained corridor, not an unconditional claim that a finite-budget forest covers every feasible passage in $\mathcal{Q}$. Coverage is controlled by the selected anchor set, subdivision depth, refinement budget, and update policy. In the conservative mode, any path retained inside the accepted conservative box union inherits the certificate above; paths that require uncovered regions must wait for additional refinement, a larger forest budget, or an explicit query-validation path.

b) Completeness and failure modes.: Proposition 2 is not a guarantee for the finite profiles used in the experiments. The implemented planner has depth, time, memory, and box-count caps; its frontiers are workload-biased; and its axis-aligned boxes can reject feasible narrow passages when envelope over-approximation, obstacle proximity, or split scheduling leaves no accepted cell under the available budget. Tolerance-gap edges, segment witnesses, online repair, and smoothing can recover some of these cases in the reported system, but they do not extend the conservative box-corridor theorem unless the resulting path is again contained in a conservative box union. Otherwise the returned path is counted only after independent query validation.

c) Reference implementation contract.: For reproduction, every reported RBF row fixes the same stage order: LECT evidence is constructed or replayed from a key containing the robot model, endpoint source, envelope representation, link radii, active-link mask, split descriptor, and depth descriptor; the scene build then validates boxes and overlap/contact edges without receiving query labels or inserting offline shortcut/segment edges; online queries anchor start/goal states, search typed graph edges, and invoke bridge repair only when graph search cannot return an audited path; finally, every returned path is strict-audited before success is counted. Cache hits may replace endpoint-envelope materialization but never bypass scene validation. Build, Online/q, final simplification, strict audit, and external-cache precomputation are separate timing buckets. The constants in table IV are profile choices for this contract, not additional assumptions in the corridor-certificate theorem.

G. Dynamic Obstacle Updates

The explicit forest representation also supports local typed-graph maintenance under obstacle edits. The update procedure assumes fixed robot kinematics and a fixed LECT topology/evidence cache; only the obstacle set and scene-dependent forest labels and typed edges change. The key distinction is asymmetric: removing obstacles can expose previously blocked cached domains, whereas inserting obstacles may invalidate boxes and segment witnesses that were previously accepted.

The update invariant is local but explicit: every box retained or inserted after an edit must pass the edited-scene validation policy, every $E_\cap$ edge must still correspond to overlap or contact of its endpoint boxes, every E_ε edge remains only a tolerance-gap query candidate, every E_s edge must keep a valid segment witness in the edited scene, and every E_π edge must retain a conservatively validated internal box chain or be removed. The update procedure is therefore a maintenance rule for the represented forest and its typed graph, not a claim of complete rediscovery of all newly available free-space regions after an edit.

For obstacle deletion, the blocker sets $\beta(C)$ recorded during the leaf-sweep stage provide a conservative candidate-selection rule for the information that was actually cached. When obstacle indices R are removed, each cached collision domain C updates its blocker set to $\beta(C) \setminus R$, with the remaining obstacle indices remapped to the edited scene. If this set becomes empty, the domain becomes eligible for promotion. Promotion either reuses sufficient cached disjointness evidence for

the edited obstacle set or reruns the standard scene check on the reconstructed envelope record. It is also filtered by containment and adjacency checks: contained candidates are discarded, disconnected candidates remain cached, and accepted candidates are inserted into the forest and connected incrementally. Thus deletion updates use the recorded blocker cache to focus validation effort and are limited to domains represented in that cache. Discovering additional newly free regions is left to a subsequent build or refinement stage.

Obstacle insertion uses the complementary path. Each added obstacle is first tested against the current live boxes, optionally restricted by the spatial dirty region. Boxes that collide with the added-obstacle checker are removed from the forest and released from the oracle reservation map; raw boxes are checked and removed by the same edited-scene logic. Segment edges whose endpoints disappear or whose waypoint witness fails the edited-scene validation are removed as well. Each invalidated box then becomes a bounded refill domain. The insertion routine performs a local leaf sweep inside this domain up to the configured insertion depth, optionally capped relative to the invalidated box depth. Leaves that pass the scene-validation policy are reinserted; leaves that remain blocked or hit the depth cap are cached with a blocker set containing the new obstacle index. The insertion update is therefore a local invalidation-and-refill operation over the typed graph: invalidated boxes are removed, local leaf-sweep refill proposes replacement boxes, affected segment witnesses are revalidated, and typed connectivity is repaired incrementally. If the edited graph still cannot answer the query, the query pipeline may use its configured repair or segment-edge mechanisms.

If no contained conservative corridor is recovered, the box-corridor certificate does not apply; repaired, post-processed, or segment-recovered paths are accepted only through query validation.

Algorithm 5 DynamicUpdate: obstacle deletion promotion and insertion refill.

Input: Forest $\mathcal{F}$, typed graph G, collision-domain cache $\mathcal{C}$, edit (Δ^-, Δ^+)
Output: Updated forest, typed graph, and collision-domain cache

if $\Delta^- \neq \emptyset$ **then**
  for all $C \in \mathcal{C}$ **do**
    remove deleted obstacle indices from $\beta(C)$ and remap surviving indices
    if $\beta(C) = \emptyset$ **then**
      promote C to $\mathcal{F}$ only if containment, adjacency, and edited-scene validation pass
    end if
  end for
  incrementally connect promoted boxes to overlapping or touching forest boxes
end if
if $\Delta^+ \neq \emptyset$ **then**
  $\mathcal{I} \leftarrow \{B \in \mathcal{F} : B$ conflicts with some obstacle in $\Delta^+\}$
  remove $\mathcal{I}$ and typed graph/segment witnesses that fail edited-scene validation
  for all $B \in \mathcal{I}$ **do**
    set the local sweep depth, optionally capped relative to depth(B)
    sweep leaves inside B to that local depth
    insert swept leaves that pass edited-scene validation and are adjacent to the surviving forest
    cache still-blocked leaves with blockers in Δ^+
  end for
  incrementally connect newly inserted boxes and revalidate surviving segment edges
end if
if the requested query is still disconnected **then**
  optionally run endpoint recovery and collision-checked segment recovery
end if
return updated $(\mathcal{F}, G, \mathcal{C})$

VI. Experiments

The experiments evaluate RBF as a reusable manipulation-planning system. They are organized as a dependency chain: the Shelf+IIWA ablation selects the RBF planning profile, the cross-algorithm and random-scene studies reuse that profile unless they explicitly sweep a budget parameter, and the dynamic-update study applies the same profile to edited scenes. All random scenes and query sets are saved before planner execution, so each method is evaluated on the same obstacle and query records. Tables and figures are generated from JSON/CSV artifacts with recorded provenance.

a) Hardware and provenance.: Wall-clock timings were collected on an Ubuntu 22.04 x86-64 workstation with an Intel Core i5-12600KF CPU and 31 GiB memory. Paper-facing RBF timing runs use one experiment process at a time and an eight-thread budget for RBF workers and threaded math libraries; OMPL baselines are run through their standard single-query interfaces unless a planner exposes its own internal parallelism. Thus the reported wall-clock tables are planner-interface comparisons under the registered protocol, not CPU-time-normalized single-thread ablations. The generated manifest records the command lines, platform metadata, thread environment, source summaries, and SHA256 hashes used to produce each table and figure.

b) Common success and timing semantics.: Planner success means that the returned path passes the fixed-resolution query-validation check. Validation time is reported separately and is not charged to planning time. Success rate is recorded over all scheduled runs; the main result tables (tables V to VII) omit the success-rate column because all selected entries are full-success points. Unless a caption states otherwise, numeric entries with available sample distributions are reported as median $[Q_1, Q_3]$; scalar-only imported context rows are reported as single values and identified by the caption. Main path columns report success-only path-length ratios. The reference is the query-level globally shortest strict-audited path whenever saved-query traces are available; scenario-level strict-audited references are used only for summary-only imported context rows and are marked in the relevant captions. This avoids silently mixing geometrically different queries. Query-time shortcut boxes and final collision shortcut boxes are disabled for the RBF rows. Main online tables define Online/q as solver time only: endpoint anchoring, online repair, graph search, or an OMPL solve call. Final simplification is fixed globally to a 10 ms OMPL budget and is not included in Online/q or

in amortized online-efficiency numbers. The independent strict audit remains reported separately. Appendix trade-off curves additionally sweep 0, 5, 10, and 50 ms simplification budgets, with 50 ms interpreted as a full-polish upper-budget setting rather than the primary online-efficiency number. Whenever a row is labeled Online/q, each query is timed as an independent solver invocation or as an explicitly recorded per-query replay; batch totals are used only for columns labeled as amortized multi-query cost.

The main experiment tables therefore report strict-audited returned-path success, not a separated conservative-box-contained success rate. Segment fraction columns are raw usage indicators for segment witnesses, and repair-dependent successes are not yet broken out as a separate result column. This distinction matches the certificate accounting in table III: conservative corridors inherit the theorem, whereas repaired, segment-witness, or performance-mode paths are counted only after the common query-validation path succeeds.

c) Cache accounting boundary.: Warm-cache RBF rows charge scene-specific build/query work plus compatible LECT lookup and evidence-read costs, but they do not include the one-time creation of the external LECT snapshot. The LECT mechanism table reports operation latency and the snapshot node/evidence counts; it should not be read as a full precompute, disk-footprint, memory-residency, or break-even study. Cache reuse is valid only when the robot model, endpoint source, envelope representation, split descriptor, link radii, active-link set, and depth descriptor match the current build. Cold diagnostic rows in table V expose the cost of recomputing deeper endpoint evidence online, while warm rows represent a deployment mode in which compatible kinematic evidence has already been materialized. A complete amortization curve that charges LECT precomputation is left outside the current artifact set.

d) RBF planning profile.: Unless otherwise stated, RBF uses the profile selected in the Shelf+IIWA ablation: a query-agnostic offline build followed by reusable online queries. The offline phase constructs an adaptive grid-partition representation over the native joint-limit space. Canonical symmetry is used only inside LECT; the planner inputs, outputs, query endpoints, and final audits remain in native joint coordinates. The selected shelf profile uses a parallel virtual leaf sweep from depth 8 to 13, d23 external endpoint evidence, budgeted grid merging, and partition-native adjacency/index construction. No start/goal, query waypoint, or query label is passed to the offline build, and the selected shelf profile does not add offline shortcut or segment edges. The online solve phase then probes the partition, applies query bridge/local repair to the five query pairs, and runs partition-native graph search; the fixed 10 ms OMPL simplification attempt is run afterward under the common final-processing protocol. The registered shelf profile fixes the b100 offline/bridge budget and keeps FINDFREEBOX binary descent with skip-to-depth 32, query-corridor paving depth 52, 12 forced bridge attempts with 1600 bridge-RRT iterations per online repair task, direct-corridor sample step 0.08 rad, and an adaptive repair cap of 24 calls. Support-hull collision uses an AABB broadphase followed by support-hull GJK narrow-phase validation. Query bridge repair is applied using the same global parameters for all five shelf query pairs. Residual segment witnesses, when inserted online after maximum-depth box repair fails, are explicitly accounted. Segment validation uses the same 0.01 rad maximum joint-space sample spacing as the final strict audit, so a segment witness cannot be accepted under a looser build-time collision check. Shelf+IIWA baseline rows additionally use the `Exp4-d23-b100` warm-cache setting, where the d23 external LECT evidence cache is read-only. In the Shelf ablation, the RBF-SH d23 row is the registered warm-cache operating point. Non-baseline diagnostic rows disable external LECT evidence unless a row is explicitly used to isolate the downstream link representation or broadphase; consequently, Link AABB and SH without broadphase retain the same d23 external evidence, while CritSample endpoints and No external LECT/HIFK-5 are cache-cold endpoint/evidence diagnostics. Random-scene rows use the same algorithm defaults, robot-local LECTDB caches, and query-independent random offline anchors rather than Shelf-specific anchors.

TABLE IV
Default RBF parameter map for reproducing the reported profiles. Values not isolated by a single-factor sweep are profile choices rather than theoretical constants.

Group	Profile values	Source
Timing/cache	8 threads; Online/q excludes simplification/audit; Shelf uses d23 read-only snapshot, while random scenes use IIWA d23 and UR5/Panda d20 LECTDB caches.	Common protocol; cache costs in tables V and X.
Offline cover	Shelf: b100 query-agnostic build, leaf sweep 8–13, no offline shortcut/segments. Random: q10x10 saved scenes, random anchors, IIWA/Panda leaf10 and UR5 leaf8.	Exp. 4 selection; saved-catalog rows in table VII.
FFB/paving	Virtual-sparse binary; Shelf skip-to-depth 32 and query paving depth 52. Random uses FFB56 for deep/refine/pave; UR5/Panda bridge start depth 8.	Generated manifests.
Anch./repair	Online endpoint anchoring; random IIWA/Panda endpoint110 and UR5 endpoint80. Shelf bridge repair uses 12 attempts, 1600 bridge-RRT iterations, 0.08 rad step, cap 24.	Robot-tuned Exp. 6 profile; Segment/BC columns in table V.
Valid./audit	AABB broadphase plus Support-Hull/GJK; segment and strict-audit spacing 0.01 rad; final OMPL simplification budget 0.01 s.	Link study in table IX; audit policy common.

A. Shelf+IIWA Leaf-Sweep–RRT Trade-off

Figure 3 and table V show the Shelf+IIWA b100 registered profile over seeds 0–7 and the five canonical query pairs. The table reports offline build time, batch-amortized online time, and build amortized over the five shelf queries at this fixed profile. CritSample rows are included only as endpoint-source diagnostics: they use the same planner and strict audit, but the envelope approximation is not a conservative-complete support-hull certificate.

The selected RBF-SH d23 row builds the reusable scene structure in 0.070 [0.070,0.071] s and answers the five-query batch with 0.032 [0.031,0.032] s amortized time after

replaying the warm d23 evidence snapshot. The No external LECT/HIFK-5 diagnostic raises build time to 7.496 [7.428,7.663] s because it charges live endpoint/evidence materialization. Thus the speed gap is cache accounting, not free computation: the warm-cache row excludes the one-time 2,097,151-record, 13,684,743,849 B (12.75 GiB) snapshot materialization/storage cost, which Table V now reports explicitly. The BC column shows only 1/40 zero-segment returned paths under this profile, so the table reports strict-audited returned-path success rather than conservative-box-contained success. The Link AABB and SH-without-broadphase rows keep the same external evidence where possible and isolate downstream representation costs: AABB loses path quality and increases raw segment fraction, while skipping the AABB broadphase more than doubles the online and amortized cost. These rows are therefore mechanism diagnostics around the registered operating point, not claims that every ablation changes only one factor under identical cache state.

B. Shelf Cross-Algorithm Context

The Shelf+IIWA cross-algorithm experiment uses the same five canonical queries and 0.01 rad final-audit setting for RBF, IRIS-NP+GCS, PRM, RRTConnect, and BIT*. RBF rows use the Exp. 4 b100 reusable-planner profile in fig. 3; table VI reports the same registered profile. For the per-query entries in table VI, the RBF online latency is measured by a separate artifact that reruns each shelf query independently from the same query-agnostic profile; it is not obtained by dividing the five-query batch run or by summing overlapping bridge-task diagnostics. The Shelf+IIWA ablation in table V separately reports reusable batched amortization. PRM is reported as a reusable build/query planner whose roadmap is shared across the five queries. RRTConnect and BIT* are one-shot online baselines with zero build time. PRM, RRTConnect, and BIT* are full-joint-space reruns under the same native-query semantics and final audit. IRIS-NP+GCS is included as an archived Shelf+IIWA contextual row whose importer checks the scene, query names, source hashes, and 0.01 rad final-audit setting. BIT* uses one timed OMPL trace per query with `samples_per_batch`=100, `rewire_factor`=5.0, `stop_on_solution_improvement`=false, a 0.5 s per-query cap, checkpoints at 0.05, 0.1, 0.2, 0.3, and 0.5 s, and the same 0.01 s final OMPL simplification budget as the other OMPL reruns. The BIT* table entry is selected from this checkpoint curve using the observed checkpoint return time, not the timeout cap; all reported paths still pass the same 0.01 rad strict final audit. This checkpoint selection is an offline quality–time context point chosen from recorded traces, not an online stopping policy that BIT* could use without hindsight; it is included to avoid comparing RBF only against a poor timeout choice. The comparison is therefore an online-latency and amortization context for the reusable box-region graph, not a claim that RBF dominates one-shot planners in path length or single-query total time.

The main Shelf+IIWA observation is that the reusable RBF build is small (0.070 s) relative to PRM's shared roadmap build (15.032 s) and the restored IRIS-NP+GCS contextual build (355.425 s), while the per-query RBF rows remain in the same latency range as stronger one-shot baselines on several queries. RRTConnect and BIT* remain important negative controls: they can be faster on easy individual queries, but their timing is not a reusable region-build cost and their Shelf paths are often longer under the same final audit.

C. Random Multi-Robot Scenes

The random-scene experiment uses a saved q10x10 prefix catalog generated before planner execution and then reused by every evaluated planner. For each robot and scene seed, the retained medium and hard scenes share the same ten query pairs, and hard strictly extends medium with additional workspace obstacles. The original easy prefixes are kept in the saved catalog as diagnostics but are not reported here because their reference paths are dominated by direct-line solutions. The generator samples actual start and goal configurations in native joint space, stores canonical information only as LECT metadata, rejects direct-line trivial cases, and uses strict collision-checked reference-planner probes to validate the saved query set. Scenes are stratified by robot (IIWA, UR5, Panda), difficulty (medium, hard), and ten saved scene seeds, giving 600 saved query evaluations per planner method. Random-scene RBF rows build one query-agnostic forest per obstacle scene and then reuse it for the ten online queries. They do not reuse Shelf+IIWA anchors or the Shelf d23 cache; UR5 and Panda use robot-local d20 LECTDB caches, while IIWA uses a d23 LECTDB root. The random-scene scan keeps the Exp. 4/Exp. 5 online protocol, audit resolution, simplification budget, and query-independent offline construction, but allows robot-local coverage profiles because the three robots have different DH geometry and split-policy behavior. The IIWA point uses the leaf10/FFB56/endpoint110 profile selected by the start-depth scan. Panda uses leaf10/FFB56/endpoint110 with FFB start depth 8 and a bounded adaptive online retry budget to recover the hard saved scenes. The UR5 point uses leaf8/FFB56/endpoint80 with the support-hull-volume split schedule and a lower query-bridge edge penalty, which reduces long detours without increasing the offline build. The table reports one readable RBF trade-off point per robot–difficulty stratum: among available full-success budget points within 8% of the shortest audited mean path, selection first minimizes Online/q, then ten-query amortized time, and finally path length; the selected profile is recorded in the manifest. Figure 5 places the registered RBF profiles against the PRM/BIT* checkpoint envelopes. PRM is built once per saved obstacle scene and queried on the same ten pairs; RRTConnect and BIT* solve each query independently as one-shot online baselines. The RRTConnect context rows use the saved

TABLE V

Shelf+IIWA RBF ablation at the selected b100 point. Numeric entries are median [Q1,Q3] and exclude the fixed 0.01 s final simplification from timing columns. Cache marks whether Build excludes replay of the precomputed d23 LECT snapshot (d23 excl.) or includes live endpoint/evidence materialization (live incl.); the d23 snapshot used by warm-cache rows contains 2,097,151 evidence records and occupies 13,684,743,849 B (12.75 GiB) on disk. $L/L^\star$ uses success-only aggregation and the query-level globally shortest strict-audited path found for the same shelf query across all available Exp. 4 ablation runs. BC is the number of strict-audited returned paths with zero recorded segment edges over all queries, used as a box-contained proxy rather than a conservative-only certificate. Boxes B/F gives build/final box-count medians. Seg. F/E gives raw segment fraction and final segment witness-edge count. Audit/q is strict audit time per query and is not charged to planning.

Case	Cache	Build	Online/q	Amort@5	$L/L^\star$	BC	Boxes B/F	Seg. F/E	Audit/q
RBF-SH d23	d23 excl.	0.070 [0.070,0.071]	0.018 [0.017,0.018]	0.032 [0.031,0.032]	1.05 [1.03,1.24]	1/40	63/345	0.17/78	0.014 [0.013,0.015]
CritSample endpoints	live incl.	0.035 [0.034,0.036]	0.020 [0.020,0.021]	0.027 [0.027,0.029]	1.06 [1.02,1.20]	1/40	51/336	0.15/61	0.014 [0.013,0.016]
Link AABB	d23 excl.	0.061 [0.061,0.062]	0.021 [0.020,0.021]	0.033 [0.033,0.033]	1.08 [1.03,1.32]	1/40	63/306	0.23/77	0.014 [0.012,0.016]
No external LECT, HIFK-5	live incl.	7.496 [7.428,7.663]	0.053 [0.051,0.054]	1.549 [1.538,1.585]	1.06 [1.03,1.28]	1/40	4/286	0.16/67	0.014 [0.014,0.015]
SH w/o broadphase	d23 excl.	0.151 [0.149,0.154]	0.049 [0.045,0.050]	0.079 [0.076,0.081]	1.05 [1.03,1.24]	1/40	63/345	0.17/78	0.014 [0.013,0.015]

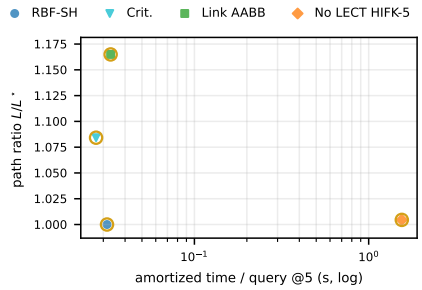


Fig. 3. Shelf+IIWA RBF ablation. Five-query amortized time versus path ratio $L/L^\star$; only full-success points are shown; gold rings mark Table V rows.

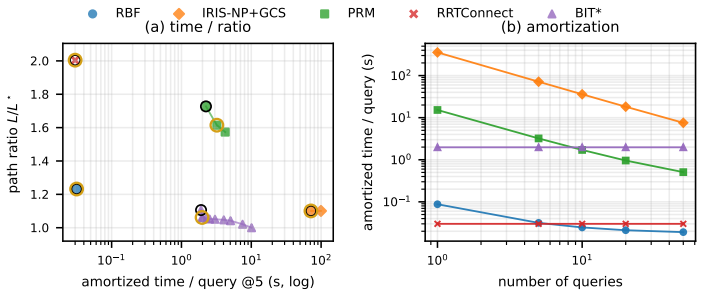


Fig. 4. Shelf+IIWA cross-algorithm trade-off. Panel (a) shows full-success amortized-time–path-ratio points; black rings mark first full-success points; panel (b) shows amortization for Table VI.

TABLE VI

Shelf+IIWA per-query cross-algorithm comparison. Rows are stratified by query. Numeric entries are median [Q1,Q3]; online solve time is in seconds and excludes the fixed 0.01 s final simplification. $L/L^\star$ uses success-only aggregation and the query-level globally shortest strict-audited path found for the same query across all available methods and checkpoints. RBF uses the registered Exp. 4 profile, with time measured by independent single-query wall-clock reruns. PRM grows one cumulative shared roadmap for the five-query batch; registered endpoint milestones and cumulative construction are charged to Build. BIT* reports the best audited incumbent along a single cumulative checkpoint trace. Full curves appear in Figure 4.

	RBF b100 Build 0.070 s		**IRIS-NP+GCS** Build 355.425 s		**PRM** Build 15.032 s		**RRTConnect** Build 0.000 s		**BIT*** Build 0.000 s	
Query	T (s)	$L/L^\star$	T (s)	$L/L^\star$	T (s)	$L/L^\star$	T (s)	$L/L^\star$	T (s)	$L/L^\star$
AS→TS	0.043 [0.041,0.046]	1.40 [1.34,1.68]	0.449 [0.449,0.449]	1.44 [1.44,1.44]	0.073 [0.029,0.170]	1.61 [1.44,1.73]	0.034 [0.024,0.052]	2.27 [2.05,2.72]	0.204 [0.204,0.204]	1.33 [1.28,1.43]
TS→CS	0.015 [0.013,0.017]	1.13 [1.11,1.14]	0.269 [0.269,0.269]	1.26 [1.26,1.26]	0.287 [0.123,0.394]	3.04 [2.40,3.50]	0.054 [0.021,0.089]	3.98 [2.80,5.09]	1.904 [1.904,1.904]	1.22 [1.18,1.63]
CS→LB	0.041 [0.039,0.043]	1.44 [1.32,1.61]	0.381 [0.381,0.381]	1.07 [1.07,1.07]	0.247 [0.184,0.338]	1.49 [1.39,2.09]	0.032 [0.027,0.070]	2.20 [1.75,2.51]	0.057 [0.054,0.064]	1.50 [1.26,1.83]
LB→RB	0.009 [0.008,0.009]	1.09 [1.07,1.22]	0.786 [0.786,0.786]	1.07 [1.07,1.07]	0.191 [0.100,0.288]	1.20 [1.10,1.28]	0.001 [0.001,0.002]	1.29 [1.27,1.32]	0.054 [0.054,0.054]	1.07 [1.03,1.10]
RB→AS	0.005 [0.005,0.005]	1.04 [1.03,1.05]	0.527 [0.527,0.527]	1.01 [1.01,1.01]	0.328 [0.253,0.400]	1.23 [1.17,1.24]	0.001 [0.001,0.001]	1.07 [1.03,1.10]	0.054 [0.054,0.055]	1.03 [1.03,1.03]

TABLE VII

Saved-catalog random-scene best trade-off points. Numeric entries are median [Q1,Q3] when distributions are available; online solve time is in seconds and excludes the fixed 0.01 s final simplification. $L/L^\star$ uses success-only aggregation with a query-level globally shortest strict-audited reference when saved-query samples are available; summary-only imported context rows use a scenario-level reference. RBF and PRM are reusable planners and report Build/Online/q/$L/L^\star$; RRTConnect and BIT* are one-shot online baselines. PRM and BIT* entries select the fastest audited checkpoint within 1.08× of the best path independently for each query/seed, rather than charging one fixed checkpoint to all queries. Full curves appear in Figure 5.

Scenario	RBF			PRM			RRTConnect		BIT*	
	Build	Online/q	$L/L^\star$	Build	Online/q	$L/L^\star$	Online/q	$L/L^\star$	Online/q	$L/L^\star$
IIWA-Medium	0.088 [0.078,0.097]	0.024 [0.021,0.040]	1.00 [1.00,1.00]	0.565 [0.016,1.528]	0.029 [0.012,0.058]	1.07 [1.05,1.11]	0.002 [0.001,0.002]	1.01 [1.00,1.04]	0.014 [0.007,0.042]	1.04 [1.01,1.06]
IIWA-Hard	0.093 [0.084,0.100]	0.032 [0.028,0.048]	1.00 [1.00,1.00]	0.721 [0.036,0.995]	0.029 [0.013,0.061]	1.07 [1.04,1.09]	0.002 [0.002,0.003]	1.03 [1.00,1.05]	0.027 [0.012,0.076]	1.04 [1.02,1.06]
UR5-Medium	0.306 [0.294,0.310]	0.063 [0.017,0.084]	1.11 [1.03,1.42]	1.836 [1.195,2.603]	0.057 [0.039,0.094]	1.01 [1.00,1.06]	0.023 [0.002,0.049]	1.22 [1.04,1.51]	0.026 [0.012,0.058]	1.03 [1.00,1.10]
UR5-Hard	0.302 [0.290,0.312]	0.074 [0.027,0.107]	1.14 [1.02,1.36]	2.684 [1.727,3.632]	0.068 [0.044,0.110]	1.00 [1.00,1.05]	0.048 [0.008,0.090]	1.29 [1.11,1.53]	0.046 [0.022,0.091]	1.05 [1.01,1.15]
PANDA-Medium	0.314 [0.283,0.353]	0.036 [0.024,0.067]	1.01 [1.00,1.06]	0.553 [0.008,1.180]	0.041 [0.022,0.073]	1.07 [1.03,1.10]	0.005 [0.002,0.011]	1.19 [1.10,1.31]	0.016 [0.007,0.031]	1.01 [1.00,1.04]
PANDA-Hard	0.280 [0.275,0.291]	0.285 [0.132,0.336]	1.03 [1.00,1.10]	1.436 [0.746,4.094]	0.044 [0.024,0.073]	1.08 [1.03,1.13]	0.056 [0.019,0.136]	1.22 [1.13,1.35]	0.062 [0.027,0.124]	1.00 [1.00,1.11]

q10x10 per-query rerun, charge only the observed solve time for Online/q, and use success-only mean path length for the normalized path column. For PRM and BIT*, the table does not charge one fixed checkpoint to every query. Instead, for each saved query/seed it selects the fastest strict-audited checkpoint whose path is within 8% of that query's best audited path, then aggregates those selected per-query records. Both PRM and BIT* use the same progressive checkpoint schedule up to a 60 s cap with quality-stall early stopping, and the table reports observed return times rather than timeout caps. This is a hindsight selection from saved traces that favors the contextual

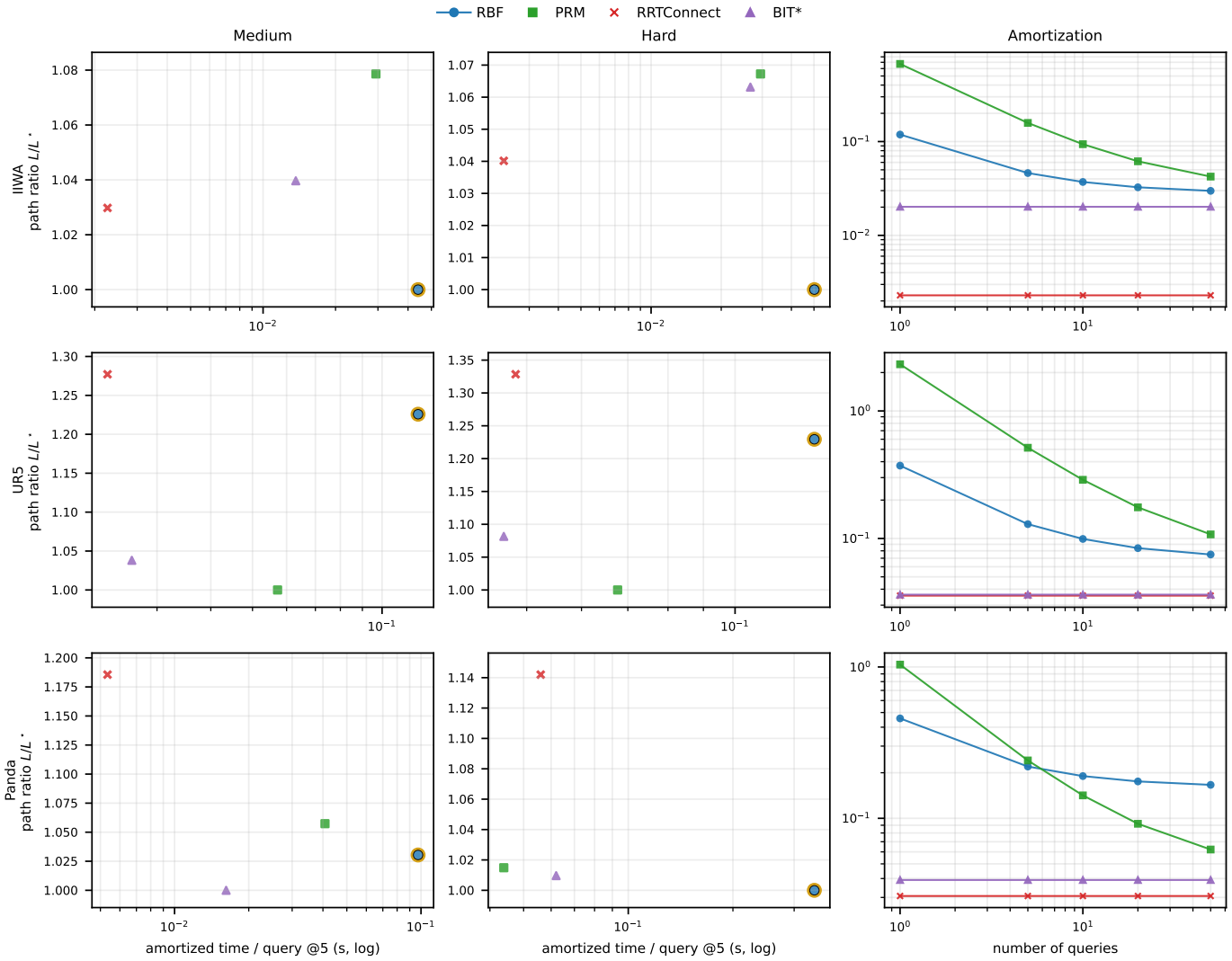


Fig. 5. Saved-catalog random-scene trade-off. Rows separate IIWA, UR5, and Panda; columns show medium, hard, and per-robot query-count amortization. The medium/hard panels plot five-query amortized time versus $L/L^\star$. Blue markers show the registered RBF profiles on the saved q10x10 prefix catalog. The amortization panels use the same per-query median solve times as the table. PRM and BIT* are saved-catalog cumulative reruns with the same 0.01 rad final audit; their reported context points are selected independently per query/seed from the cumulative checkpoints. Only 100% success-rate points are plotted; black rings mark the first full-success point on each curve and gold rings mark the tabulated trade-off point. Context rows for non-RBF reusable planners are shown when available under the same saved-scene protocol.

PRM/BIT* rows; it should be read as a quality-normalized baseline envelope rather than as a deployable online stopping rule. IRIS-NP+GCS is not included in the random-scene table because its available saved-catalog runs did not achieve sufficiently reliable strict-audit success across the q10x10 random scenes; we therefore keep it only in the Shelf+IIWA contextual comparison where the archived Shelf protocol is directly comparable. These results should be read as a reusable-planner study: RBF's advantage is expected to appear in online reuse and amortized multi-query cost, while BIT* and RRTConnect can still be stronger one-shot planners on individual random queries.

Across the selected full-success RBF points, the random-scene build IQRs range from 0.078–0.097 s for IIWA-Medium to 0.287–0.304 s for UR5-Medium, while online-query IQRs range from 0.016–0.083 s for UR5-Medium to 0.132–0.400 s for Panda-Hard. Panda-Hard is the clearest stress case for the registered profile: the 7-DOF arm, harder prefix obstacles, and stricter saved query set increase the chance that endpoint anchoring, online fallback, or bridge repair rather than pure graph lookup dominates Online/q. The current table does not decompose those costs by failure cause, so this is treated as a limitation of the reported artifact rather than as a tuned scaling claim. The table also shows the limitation of the design point: for many individual random queries, RRTConnect or BIT* has lower online latency because it avoids building a reusable region graph. The intended benefit of RBF is therefore the combination of a low-cost reusable build, full-success saved-catalog execution, and repeated-query reuse, not a universal single-query speed claim.

D. Mechanism Studies

TABLE VIII
Endpoint envelope source study. Vol. is median envelope volume; Time is construction only; Neg. gap uses the sampling-union reference.

Width	Source	Safe	Vol.	Time (μs)	Neg. gap
0.02	IFK_AA	Y	4.45e−5	5.48	0
0.02	HIFK_3	Y	4.25e−5	25.94	0
0.02	HIFK_5	Y	4.19e−5	70.40	0
0.02	CritSample	N	4.09e−5	8.68	−4.8e−5
0.02	Analytical	Y	4.09e−5	6429.96	0
0.02	MC	N	2.61e−5	2251.74	−6.1e−3
0.10	IFK_AA	Y	7.71e−3	6.45	0
0.10	HIFK_3	Y	6.19e−3	29.16	0
0.10	HIFK_5	Y	5.75e−3	73.82	0
0.10	CritSample	N	5.09e−3	10.03	−1.2e−3
0.10	Analytical	Y	5.09e−3	6602.25	0
0.10	MC	N	3.74e−3	11233.26	−3.3e−2
0.50	IFK_AA	Y	5.60e0	7.35	0
0.50	HIFK_3	Y	2.03e0	29.38	0
0.50	HIFK_5	Y	1.35e0	72.32	0
0.50	CritSample	N	6.40e−1	16.33	−3.1e−2
0.50	Analytical	Y	6.51e−1	7096.01	−4.5e−3
0.50	MC	N	5.34e−1	56835.57	−1.2e−1

TABLE IX
Link envelope representation study. Env. is construction time; Coll. is obstacle-collision time.

Width	Envelope	Splits	Vol.	Env. (μs)	Coll. (μs)
0.02	LinkIAABB	1	0.013	0.110	0.300
0.02	SupportHull	1	0.000	0.166	0.309
0.05	LinkIAABB	1	0.020	0.139	0.310
0.05	SupportHull	1	0.003	0.177	0.306
0.10	LinkIAABB	1	0.038	0.141	0.297
0.10	SupportHull	1	0.014	0.188	0.305
0.20	LinkIAABB	1	0.147	0.136	0.305
0.20	SupportHull	1	0.105	0.171	0.311
0.30	LinkIAABB	1	0.502	0.137	0.305
0.30	SupportHull	1	0.440	0.175	4.823
0.50	LinkIAABB	1	4.514	0.143	0.304
0.50	SupportHull	1	4.448	0.184	10.950

TABLE X
LECT snapshot/cache operation costs on the persisted d23 snapshot.

Operation	Ops	Avg. (μs)	Nodes	Evidence
Open snapshot	1	86.219	2097151	2097151
Node box	20000	1.084	2097151	2097151
Exact lookup	20000	0.849	2097151	2097151
Endpoint lookup	20000	1.860	2097151	2097151
Endpoint lookup (hot)	20000	0.577	2097151	2097151
Range query	20000	22.596	2097151	2097151
Evidence read	20000	1.262	2097151	2097151
Evidence read (hot)	20000	0.132	2097151	2097151

Tables VIII–X isolate source certificates, envelope narrow phases, and warm LECT replay; the d23 hot endpoint/evidence reads are 0.577/0.132 μs.

E. Dynamic Updates

TABLE XI
Adaptive leaf-sweep maintenance only, not end-to-end replanning. Times are seconds shown as $[Q_1, Q_3]$ over saved ordered random scenes. Warm@10 and Warm@15 are fresh adaptive leaf-sweep builds; Insert and Remove are batched updates between the two obstacle counts. Speedup is Warm@15/Update.

Warm@10	Insert	Ins. sp.	Remove	Rem. sp.	Warm@15
$[1.356, 1.626]$	$[0.031, 0.050]$	35.9–55.9×	$[0.034, 0.071]$	24.5–47.7×	$[1.571, 1.800]$

The dynamic-update study isolates scene-cache maintenance. Each seed builds a two-obstacle adaptive leaf-sweep partition, inserts the next saved AABB obstacle, and removes it again using the same virtual-topology LECT replay path as the planner; no query bridge, connector, OMPL simplification, or final audit is run. Insertions invalidate dirty boxes and refill local LECT subtrees, whereas deletions mainly promote cached collision domains. The measured 0.031–0.050 s insert and 0.034–0.071 s removal costs are 35.9–55.9× and 24.5–47.7× faster than the corresponding three-obstacle warm rebuilds, supporting local maintenance but not claiming an end-to-end query speedup.

VII. Conclusion

This paper presented RBF, a link-envelope-guided box-forest planner for multi-query manipulation. It lifts endpoint-to-link swept-capsule bounds to C-space box validation, grows accepted boxes into a scene-level corridor graph, and separates scene labels from LECT's kinematics-keyed evidence cache. The certificate is conditional: it applies to paths inside conservatively validated box unions, while repaired, post-processed, and segment-witness paths must pass query validation. Under this policy, RBF gives a 0.070 [0.070,0.071] s Shelf+IIWA reusable build, 0.032 [0.031,0.032] s five-query amortized time, full-success selected saved random-scene points, microsecond-scale warm LECT replay, and 24.5–55.9× isolated local-update speedups. The limitations are explicit: one-shot planners can be faster on easy individual queries; rich convex regions can be preferable under larger amortization workloads; and self-collision, attached objects, grippers, and task-specific forbidden regions must be added to the validation policy before they inherit the certificate. Future work will study full cache-precompute amortization, end-to-end update-and-replan timing, and containment-preserving smoothing.

References

[1] R. Deits and R. Tedrake, "Computing large convex regions of obstacle-free space through semidefinite programming," in *Algorithmic Foundations of Robotics XI*, 2015, pp. 109–124.

[2] T. Marcucci, M. Petersen, D. von Wrangel, and R. Tedrake, "Motion planning around obstacles with convex optimization," *Sci. Robot.*, vol. 8, no. 84, 2023.

[3] T. Marcucci, J. Umenberger, P. Parrilo, and R. Tedrake, "Shortest paths in graphs of convex sets," *SIAM J. Optim.*, vol. 34, no. 1, pp. 507–532, 2024.

[4] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006.

[5] ——, "Rapidly-exploring random trees: A new tool for path planning," Iowa State University, Tech. Rep. TR 98-11, 1998.

[6] L. E. Kavraki, P. Švestka, J.-C. Latombe, and M. H. Overmars, "Probabilistic roadmaps for path planning in high-dimensional configuration spaces," *IEEE Trans. Robot. Autom.*, vol. 12, no. 4, pp. 566–580, 1996.

[7] S. Karaman and E. Frazzoli, "Sampling-based algorithms for optimal motion planning," *Int. J. Robot. Res.*, vol. 30, no. 7, pp. 846–894, 2011.

[8] A. Amice, H. Dai, P. Werner, A. Zhang, and R. Tedrake, "Finding and optimizing certified, collision-free regions in configuration space for robot manipulators," in *Algorithmic Foundations of Robotics XV*. Springer, 2022, pp. 328–348.

[9] M. Petersen and R. Tedrake, "Growing convex collision-free regions in configuration space using nonlinear programming," 2023. [Online]. Available: https://arxiv.org/abs/2303.14737

[10] P. Werner, A. Amice, T. Marcucci, D. Rus, and R. Tedrake, "Approximating robot configuration spaces with few convex sets using clique covers of visibility graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 10 359–10 365.

[11] D. Blackmore and M. C. Leu, "Analysis of swept volume via Lie groups and differential equations," *Int. J. Robot. Res.*, vol. 11, no. 6, pp. 516–537, 1992.

[12] K. Abdel-Malek, J. Yang, D. Blackmore, and K. Joy, "Swept volumes: foundation, perspectives, and applications," *Int. J. Shape Model.*, vol. 12, no. 1, pp. 87–127, 2006.

[13] S. Gottschalk, M. C. Lin, and D. Manocha, "OBBTree: A hierarchical structure for rapid interference detection," in *ACM SIGGRAPH*, 1996.

[14] E. Larsen, S. Gottschalk, M. C. Lin, and D. Manocha, "Fast proximity queries with swept sphere volumes," University of North Carolina at Chapel Hill, Tech. Rep. TR99-018, 1999.

[15] E. G. Gilbert, D. W. Johnson, and S. S. Keerthi, "A fast procedure for computing the distance between complex objects in three-dimensional space," *IEEE J. Robot. Autom.*, vol. 4, no. 2, pp. 193–203, 1988.

[16] G. v. d. Bergen, "A fast and robust GJK implementation for collision detection of convex objects," *J. Graph. Tools*, vol. 4, no. 2, pp. 7–25, 1999.

[17] J. Pan, S. Chitta, and D. Manocha, "FCL: A general purpose library for collision and proximity queries," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2012, pp. 3859–3866.

[18] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to Interval Analysis*. SIAM, 2009.

[19] L. Jaulin, M. Kieffer, O. Didrit, and E. Walter, *Applied Interval Analysis*. Springer, 2001.

[20] J. M. Snyder, "Interval analysis for computer graphics," in *Proc. ACM SIGGRAPH*, 1992, pp. 121–130.

[21] J.-P. Merlet, "Solving the forward kinematics of a Gough-type parallel manipulator with interval analysis," *Int. J. Robot. Res.*, vol. 23, no. 3, pp. 221–235, 2004.

[22] ——, "Interval analysis for certified numerical solution of problems in robotics," *Int. J. Appl. Math. Comput. Sci.*, vol. 19, no. 3, pp. 399–412, 2009.

[23] S. Redon, Y. J. Kim, M. C. Lin, and D. Manocha, "Interactive and continuous collision detection for avatars in virtual environments," in *IEEE Virtual Reality Conference*, 2004, pp. 117–124.

[24] S. Redon, M. C. Lin, D. Manocha, and Y. J. Kim, "Fast continuous collision detection for articulated models," *J. Comput. Inf. Sci. Eng.*, vol. 5, no. 2, pp. 126–137, 2005.

[25] X. Zhang, S. Redon, M. Lee, and Y. J. Kim, "Continuous collision detection for articulated models using Taylor models and temporal culling," *ACM Trans. Graph.*, vol. 26, no. 3, p. 15, 2007.

[26] F. Schwarzer, M. Saha, and J.-C. Latombe, "Adaptive dynamic collision checking for single and multiple articulated robots in complex environments," *IEEE Trans. Robot.*, vol. 21, no. 3, pp. 338–353, 2005.

[27] J. A. Corrales, F. A. Candelas, and F. Torres, "Safe human–robot interaction based on dynamic sphere-swept line bounding volumes," *Robot. Comput.-Integr. Manuf.*, vol. 27, no. 1, pp. 177–185, 2011.

[28] R. Bohlin and L. E. Kavraki, "Path planning using lazy PRM," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 521–528.

[29] C. M. Dellin and S. S. Srinivasa, "A unifying formalism for shortest path problems with expensive edge evaluations via lazy best-first search over paths with edge selectors," in *International Conference on Automated Planning and Scheduling (ICAPS)*, 2016.

[30] E. Schmerling, L. Janson, and M. Pavone, "Optimal sampling-based motion planning under differential constraints: The drift-less case," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015.

[31] D. Coleman, I. Sucan, S. Chitta, and N. Correll, "Reducing the barrier to entry of complex robotic software: A MoveIt! case study," *J. Softw. Eng. Robot.*, vol. 5, no. 1, pp. 3–16, 2014.

[32] T. Lozano-Pérez, "Spatial planning: A configuration space approach," *IEEE Trans. Comput.*, vol. C-32, no. 2, pp. 108–120, 1983.

[33] R. A. Brooks and T. Lozano-Pérez, "A subdivision algorithm in configuration space for findpath with rotation," *IEEE Trans. Syst. Man Cybern.*, vol. SMC-15, no. 2, pp. 224–233, 1985.

[34] J. J. Kuffner and S. M. LaValle, "RRT-connect: An efficient approach to single-query path planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, pp. 995–1001.

[35] J. D. Gammell, S. S. Srinivasa, and T. D. Barfoot, "Batch informed trees (BIT*): Sampling-based optimal planning via the heuristically guided search of implicit random geometric graphs," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2015, pp. 3067–3074.

[36] J. D. Gammell, T. D. Barfoot, and S. S. Srinivasa, "Informed sampling for asymptotically optimal path planning," *IEEE Trans. Robot.*, vol. 34, no. 4, pp. 966–984, 2018.

[37] L. Janson, E. Schmerling, A. Clark, and M. Pavone, "Fast marching tree: A fast marching sampling-based method for optimal motion planning in many dimensions," *Int. J. Robot. Res.*, vol. 34, no. 7, pp. 883–921, 2015.

[38] I. A. Şucan, M. Moll, and L. E. Kavraki, "The open motion planning library," *IEEE Robot. Autom. Mag.*, vol. 19, no. 4, pp. 72–82, 2012.

[39] M. Moll, I. A. Şucan, and L. E. Kavraki, "Benchmarking motion planning algorithms: An extensible infrastructure for analysis and visualization," *IEEE Robot. Autom. Mag.*, vol. 22, no. 3, pp. 96–102, 2015.

[40] N. Ratliff, M. Zucker, J. A. Bagnell, and S. Srinivasa, "CHOMP: Gradient optimization techniques for efficient motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2009, pp. 489–494.

[41] M. Kalakrishnan, S. Chitta, E. Theodorou, P. Pastor, and S. Schaal, "STOMP: Stochastic trajectory optimization for motion planning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2011, pp. 4569–4574.

[42] J. Schulman, Y. Duan, J. Ho, A. Lee, I. Awwal, H. Bradlow, J. Pan, S. Patil, K. Goldberg, and P. Abbeel, "Motion planning with sequential convex optimization and convex collision checking," *Int. J. Robot. Res.*, vol. 33, no. 9, pp. 1251–1270, 2014.

[43] M. Mukadam, J. Dong, X. Yan, F. Dellaert, and B. Boots, "Continuous-time gaussian process motion planning via probabilistic inference," *Int. J. Robot. Res.*, vol. 37, no. 11, pp. 1319–1340, 2018.